

Resúmenes — Criptografía y Seguridad (72.44)

Segundo Parcial
ITBA

Material de estudio basado en clases, transcripts, clase de consulta y parciales

Índice

1. Políticas y Control de Acceso	5
1.1. Conceptos base	5
1.1.1. ¿Qué es seguridad? Política y estados	5
1.1.2. La tríada CIA	5
1.1.3. Sujeto, Objeto, Acceso	6
1.1.4. Modelo de seguridad	6
1.2. Modelos de Confidencialidad: Bell-LaPadula (BLP)	6
1.2.1. Las dos reglas (+ la síntesis)	7
1.2.2. Compartimentos y retículo (Lattice)	8
1.3. Modelos de Integridad: Biba y Clark-Wilson	9
1.3.1. Modelo Biba	9
1.3.2. Modelo Clark-Wilson	10
1.3.3. Más allá de los modelos teóricos	10
1.4. Control de Acceso: clasificación	10
1.4.1. Control discrecional por extensión	11
1.4.2. Control discrecional por comprensión	12
1.5. Resolución de conflictos entre reglas	12
1.5.1. Reglas basadas en contexto	13
1.5.2. Open Policy Agent (OPA)	13
1.6. Ejemplos reales de control de acceso	13
1.6.1. OAuth 2.0 (Open Authorization)	14
1.7. Metodología: diseño de compartimentos (ejercicio estrella)	14
1.7.1. Chinese Wall / Muralla China	15
1.8. Mentalidad de seguridad (clase de consulta)	15
2. Autenticación	17
2.1. Qué es autenticar	17
2.1.1. Componentes de un sistema de autenticación	17
2.2. Factores de autenticación	18
2.2.1. Biometría	19
2.3. Claves (passwords)	20
2.3.1. Almacenamiento de claves	20
2.3.2. Dispositivo criptográfico (HSM) y token físico	21
2.4. Ataques a un sistema de autenticación	21
2.4.1. Prevenciones	21
2.5. Fuerza bruta: fórmula de Anderson y metodología	22
2.6. Salt (salting)	23
2.6.1. Funciones lentas: PBKDF2 (PKCS 5 v2.0)	24
2.7. Challenge-Response y EKE	25

2.8.	OTP — Claves de uso único	26
2.8.1.	HOTP — RFC 4226 (basado en HMAC)	26
2.8.2.	TOTP — RFC 6238 (basado en tiempo)	26
2.9.	Multifactor y multicanal	26
2.10.	Autenticación remota (SSO / Single Sign-On)	27
2.11.	Lectura recomendada	28
3.	Principios de Diseño de Seguridad y Vulnerabilidades	29
3.1.	Introducción: qué son los principios de diseño	29
3.2.	Principios de Saltzer & Schroeder (1975)	29
3.2.1.	1. Mínimo privilegio (Least privilege)	30
3.2.2.	2. Fallar de forma segura (Fail-safe defaults)	30
3.2.3.	3. Simplicidad (Economy of mechanism)	30
3.2.4.	4. Mediación completa (Complete mediation)	31
3.2.5.	5. Sistema/Diseño abierto (Open design)	31
3.2.6.	6. Separación / Segregación de privilegios (Separation of privilege)	31
3.2.7.	7. Mecanismos exclusivos (Defense-in-Depth)	32
3.2.8.	8. Menor asombro / Aceptación psicológica (Psychological acceptability)	32
3.3.	La seguridad NO se garantiza	33
3.4.	Calidad/Procesos: DevOps vs DevSecOps vs MLOps	33
3.5.	Modelado de amenazas (Threat Modeling)	33
3.5.1.	Modelo STRIDE	34
3.5.2.	DFD (Data Flow Model) + STRIDE	34
3.6.	Amenazas y Vulnerabilidades	35
3.6.1.	Tipos de amenazas (taxonomías)	36
3.6.2.	Tipos de vulnerabilidades	36
3.6.3.	MITRE: CVE, CWE y Zero-day	37
3.7.	Buffer overflow e inyección de código	37
3.8.	Vulnerabilidades web	38
3.8.1.	SQL Injection (CWE-89)	38
3.8.2.	Cross-Site Scripting / XSS (CWE-79)	39
3.8.3.	CSRF (Cross-Site Request Forgery)	39
3.8.4.	OS Command / Shell Injection (CWE-78)	40
3.8.5.	Path / Directory Traversal	40
3.8.6.	XXE (XML External Entities)	40
3.8.7.	Organizaciones de referencia (OWASP)	40
3.9.	Malware básico (mencionado / referenciado para definir)	40
3.10.	La idea del “chip paranoico”	41
3.11.	Lectura recomendada	41
4.	Flujo de Información y Malware	42
4.1.	Flujo de información: el problema	42
4.2.	Confinamiento y aislación total	43
4.3.	Canales encubiertos y canales laterales	43
4.4.	Entropía como medida de información	45
4.5.	Flujo de información vía entropía	46
4.5.1.	Flujo directo	46
4.5.2.	Flujo indirecto	47
4.5.3.	Límites (compartidos con la verificación formal)	47
4.6.	Control de flujo: compilación vs. ejecución	47
4.6.1.	Aislamiento (en ejecución)	48
4.7.	Malware	49
4.7.1.	Tipos de malware	49

4.7.2. Ciclo de ataque del malware (Malware attack cycle)	50
5. Seguridad en la Empresa	52
5.1. Gobernanza (Governance)	52
5.2. Gestión de riesgos (Risk Management)	52
5.2.1. Identificación del riesgo	53
5.2.2. Evaluación del riesgo	53
5.2.3. Mitigación del riesgo — las 5 opciones	53
5.2.4. Monitoreo	53
5.3. Políticas de seguridad	54
5.4. Estrategias: Defense-in-depth vs. Zero Trust	54
5.4.1. Defense-in-depth (defensa en profundidad)	54
5.4.2. Zero Trust Architecture (ZTA)	57
5.5. Privileged Access Management (PAM)	58
5.6. Prevención de vulnerabilidades: DevSecOps	59
5.7. Protección de la nube	60
5.8. Evaluaciones de seguridad	60
5.8.1. Penetration Test (PenTest)	61
5.8.2. Red / Blue / Purple Team	61
5.8.3. Breach and Attack Simulation (BAS)	62
6. Pentesting	63
6.1. Formas de verificar un sistema: verificación formal vs. pentesting	63
6.2. Objetivos y marco ético (ethical hacking)	64
6.3. Metodología informal (preparación)	65
6.4. Metodología de Hipótesis de Falla (LOS PASOS)	65
6.4.1. Paso 1: Recolección de información	66
6.4.2. Paso 2: Hipótesis	67
6.4.3. Paso 3: Prueba	68
6.4.4. Paso 4: Generalización	68
6.4.5. Paso 5: Eliminación (opcional)	69
6.5. Herramientas y técnicas mencionadas	69
6.6. Ejemplos	69
6.6.1. Ejemplo 1: Michigan Terminal System (MTS)	69
6.6.2. Ejemplo 2: Ataque externo (ingeniería social)	70
6.7. Limitaciones: el pentesting NO garantiza la seguridad	71
6.8. Lectura recomendada	71
7. Protección de Datos	72
7.1. El problema y el ciclo de protección	72
7.1.1. Data Governance	72
7.1.2. Data Roles (jerarquía)	73
7.2. Marco legal y regulaciones	73
7.2.1. Regulación argentina: Ley 25.326	73
7.2.2. Regulaciones internacionales	74
7.3. Estados del dato y encriptación	75
7.3.1. Enforcement (Compiler-time)	75
7.3.2. Encriptación at-rest	75
7.3.3. Encriptación in-transit	76
7.3.4. Enforcement (Execution-time)	76
7.3.5. Encriptación en memoria (TME / TME-MK)	76
7.3.6. SoC / IoT / Embedded Security	76
7.4. Cumplimiento, detección y respuesta	76

7.4.1.	Comply (Cumplimiento)	76
7.4.2.	Detect – DLP (Data Loss Prevention)	77
7.4.3.	Respond (Respuesta a incidentes)	77
7.5.	Encriptación Homomórfica	77

1. Políticas y Control de Acceso

► En el parcial

Esta es la **unidad más evaluada** del segundo parcial: aparece en casi todos los exámenes. Los temas estrella son:

- **Diseño de compartimentos** (construir un retículo a partir de un enunciado en lenguaje natural). Confidencialidad (Bell-LaPadula) e integridad (Biba/Clark-Wilson).
- Identificar la política dado un retículo, y nombrarla bien (BLP vs Biba).
- Resolución de conflictos entre reglas (first-match, best-match, deny-over-allow, grant-all).
- Distinguir **modelo** / **política** / **reglas** de seguridad.
- Preguntas conceptuales V/F (matriz de acceso, ACL vs Capabilities).
- Ejercicios abiertos tipo “sos el CSO”: elegir objetivo de seguridad, modelo, política y reglas.

1.1. Conceptos base

1.1.1. ¿Qué es seguridad? Política y estados

No hay una única definición de seguridad. Igual que en criptografía cada prueba daba una definición distinta de seguridad, al pasar a sistemas dinámicos tampoco hay una definición única. La definición genérica que se usa es:

Definición: Sistema seguro

Si podemos clasificar todos los estados de un sistema en **seguros** e **inseguros**, un sistema es seguro mientras sus estados queden dentro de la política de seguridad (mientras transicione solo por estados que consideramos seguros).

Definición: Política de seguridad

Es la definición de qué estados de un sistema vamos a considerar seguros versus el resto. En un contexto (aplicación, servicio, empresa) la política define los modos y usos que se consideran seguros.

Dos conceptos potentes de esta definición:

1. Necesitamos poder caracterizar, “sacando una foto” del estado actual, si es seguro o no.
2. Un sistema que pasa a un estado inseguro **deja de ser seguro, aunque después vuelva** a un estado seguro. Por eso, ante un servidor comprometido, las buenas prácticas dicen reinstalarlo desde cero: es muy difícil volver a un estado de confianza.

1.1.2. La tríada CIA

- **Confidencialidad:** atañe a la disseminación de información. Hay información accesible a cierto grupo y no a otro.
- **Integridad:** tiene que ver con relaciones de confianza. Garantizar/certificar la calidad de algo (que una transacción ocurre, que los datos son correctos, que el origen es quien dice ser).
- **Disponibilidad:** capacidad de acceder a información o ejecutar una acción en el momento dado. Involucra temporalidad, es dinámica y por eso no se estudia tanto en criptografía (que era estática).

1.1.3. Sujeto, Objeto, Acceso

Definición: Sujeto / Objeto / Acceso

Sujeto: entidad que ejecuta cosas sobre otras entidades; es el origen de un estímulo. Puede ser persona, servicio, aplicación, servidor.

Objeto: destino de esos estímulos. Generalmente recursos: datos, servicios, bases de datos, tablas, programas, archivos.

Acceso: las formas que tienen los sujetos de interactuar con el objeto (leer, escribir, borrar, mover, ejecutar...).

Ejemplo: en Linux los sujetos son usuarios, los objetos son archivos (todo es un archivo) y los accesos son permisos. La tríada se reduce a esta base:

- **Confidencialidad:** existen sujetos que NO pueden acceder a ciertos objetos.
- **Integridad:** existen objetos que solo pueden ser modificados por ciertos sujetos confiables.
- **Disponibilidad:** existen objetos que pueden ser accedidos por ciertos sujetos cuando lo requieran.

Identidad del sujeto: lo que identifica unívocamente al sujeto dentro del universo de posibles sujetos. Es requisito para la autenticación (ej. SID en Windows, UID en `/etc/passwd`).

1.1.4. Modelo de seguridad

Definición: Modelo de seguridad

Modelo formal y lógico que define las reglas e interacciones que permiten implementar un mecanismo de control de acceso. Define qué estados son seguros y cuáles no.

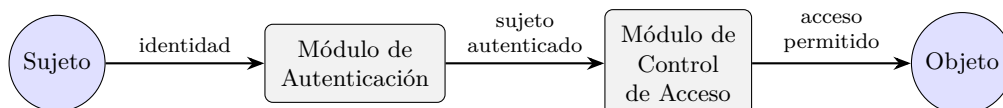
- Los modelos suelen estar **especializados**: hay modelos de confidencialidad, de integridad e incluso de disponibilidad.
- Evitan “reinventar la rueda” (resolver artesanalmente sujeto por sujeto es complejo, propenso a errores y difícil de demostrar).
- **No alcanzan por sí solos en un sistema real**: son la base de muchas implementaciones, pero rara vez se aplican enteramente.

! Importante — remarcado en clase

El profesor remarca mucho la distinción **MODELO / POLÍTICA / REGLAS**:

- **Modelo:** marco formal y lógico (ej. Bell-LaPadula, Biba, Clark-Wilson).
- **Política:** definición de qué estados son seguros para un contexto concreto (ej. “no read up, no write down”).
- **Reglas:** las reglas concretas que implementan la política (ej. las ACL/RuBAC que se cargan).

Un ejercicio típico (2025-2Q) pide identificar explícitamente cada nivel.



*Encadenamiento de módulos (ejercicio de parcial 2013): el sujeto pasa primero por la **autenticación** (¿quién sos?) y luego por el **control de acceso** (¿podés?) antes de llegar al objeto.*

1.2. Modelos de Confidencialidad: Bell-LaPadula (BLP)

Diseñado por Bell y LaPadula (IBM) para el ámbito militar, focalizado en **confidencialidad**. Fue el primer modelo público aprobado para uso en organismos gubernamentales de EE.UU.

(análogo a DES en su rol histórico). Plantea **niveles de seguridad** ordenados; el original usa *Top Secret*, *Secret*, *Confidential*, *Unclassified*, pero admite cualquier secuencia ordenada. Toda la información y todos los sujetos se clasifican en uno de esos niveles.

1.2.1. Las dos reglas (+ la síntesis)

Definición: Condición de Seguridad Simple (ss-property / CSS)

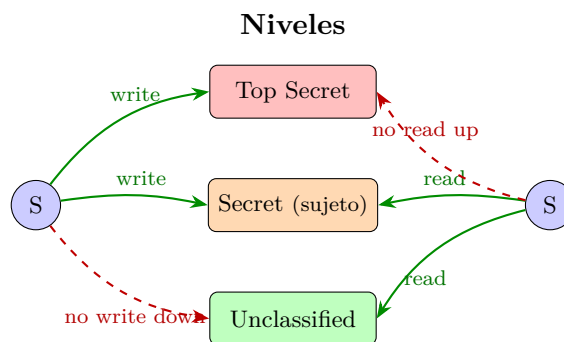
Un sujeto puede **leer** un objeto si y solo si el nivel del sujeto es igual o superior al del objeto. Se traduce como “**no read up**” (leer hacia abajo).

Definición: Regla de cierre / propiedad estrella (*-property)

Un sujeto puede **escribir** un objeto si y solo si el nivel del objeto es igual o superior al del sujeto. Se traduce como “**no write down**” (escribir hacia arriba).

Síntesis (strong star): una persona que necesita leer *y* escribir un objeto solo puede hacerlo con objetos de **su misma clasificación** (por lectura solo accede a su nivel o menores, por escritura solo a su nivel o superiores; la intersección es el mismo nivel).

BLP (confidencialidad): READ DOWN, WRITE UP.



Bell-LaPadula: un sujeto lee hacia abajo (su nivel y menores) y escribe hacia arriba (su nivel y mayores). La intersección lectura $\cap$ escritura es su mismo nivel (strong star).

¿Por qué la *-property? La CSS sola solo resuelve el acceso *directo*. La *-property garantiza **control de flujo de información**: impide que alguien con acceso alto vuelque la información a un nivel más bajo. Ejemplo del editor de texto que guarda una copia temporal: el archivo original queda protegido, pero la copia podría quedar legible para quien no debe. Ejemplo del directivo que “sin querer” filtra información que conoce al hablar con gente de nivel inferior. Caso extremo: en el Proyecto Manhattan se aisló físicamente a los científicos en un pueblo para evitar interacción con niveles inferiores (aplicación de BLP).

! Importante — remarcado en clase

“La idea de este modelo, de alguna manera, es garantizarnos la confidencialidad pero no desde un punto de vista estático: lo que nos garantiza es un control de flujo de información.”

— dicho en clase BLP tiene demostración formal (sobre máquinas de Turing, en el libro de

Bishop) de que no hay ningún camino de información que contravenga sus reglas. Eso es muy fuerte: cuando la información fluye, se copia, se fragmenta y se vuelve a juntar, es muy difícil garantizar la confidencialidad.

Limitación: principio de tranquilidad. BLP asume que la clasificación de sujetos y objetos **no cambia en el tiempo**, lo cual es falso (gente que asciende, información que se hace pública). En la práctica se agregan mecanismos al costado para reclasificar (ej. los avisos “declassified for use” en mails de organismos; se complementa con Clark-Wilson). Es un modelo muy estricto, raro de aplicar completo a todo un sistema; se reserva para las partes más críticas.

1.2.2. Compartimentos y retículo (Lattice)

Solo con 4 niveles el modelo es demasiado simple. Se agrega otra dimensión: además del nivel de seguridad, una **lista de etiquetas** (categorías) asignables a cada sujeto/objeto. Implementa el **need-to-know**: si un general se encarga de criptografía, no tiene por qué conocer los secretos nucleares, sin importar su nivel.

Definición: Compartimento

Un par formado por un **nivel** de seguridad y una **lista de etiquetas**: $(\text{nivel}, \{\text{etiquetas}\})$.

La comparación “nivel superior/inferior” se reemplaza por **dominancia**:

Definición: Dominancia

A **domina** a B ($A \succeq B$) si y solo si se cumplen **ambas** condiciones:

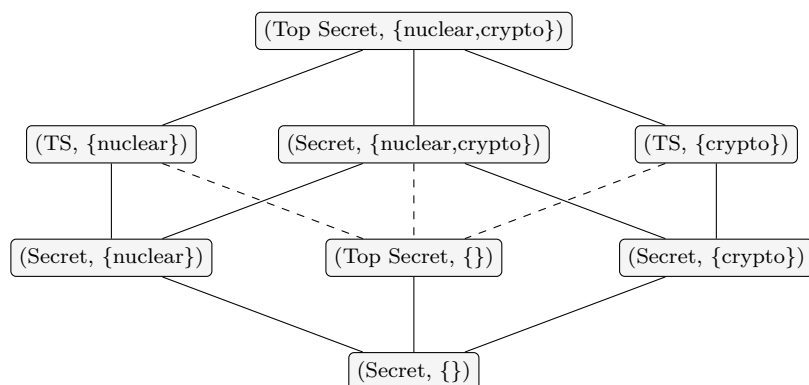
1. $\text{nivel}(A) \geq \text{nivel}(B)$, y
2. $\text{etiquetas}(B) \subseteq \text{etiquetas}(A)$ (las etiquetas de A contienen a las de B).

Reglas con compartimentos:

- Un sujeto puede **leer** un objeto si el compartimento del **sujeto domina** al del objeto.
- Un sujeto puede **escribir** un objeto si el compartimento del **objeto domina** al del sujeto.

Esto forma un **retículo (lattice)**, que es un **conjunto de orden parcial**: no todos los compartimentos son comparables. Ejemplos del libro:

- $(\text{Top Secret}, \{\text{nuclear}\})$ domina a $(\text{Secret}, \{\text{nuclear}\})$.
- $(\text{Secret}, \{\text{nuclear}\})$ domina a $(\text{Secret}, \{\})$.
- $(\text{Top Secret}, \{\text{nuclear}, \text{crypto}\})$ domina a $(\text{Top Secret}, \{\text{nuclear}\})$.
- $(\text{Secret}, \{\text{nuclear}\})$ y $(\text{Secret}, \{\text{crypto}\})$ **NO** tienen relación: mismo nivel, pero ninguna lista de etiquetas es subconjunto de la otra. No hay dominancia en ningún sentido $\Rightarrow$ están efectivamente separados (ni lectura ni escritura entre ellos).



*Retículo (diagrama de Hasse) del libro: cada arista sube hacia el que **domina**. $(\text{Secret}, \{\text{nuclear}\})$ y $(\text{Secret}, \{\text{crypto}\})$ **no son comparables** (ninguna lista de etiquetas contiene a la otra) $\Rightarrow$ están separados.*

A nivel implementación es simple: a cada objeto y a cada sujeto se le almacena como metadata un nivel y una lista de etiquetas; resolver cada operación con las dos reglas es directo.

△ Cuidado — error típico

El error más penalizado de la unidad: **confundir las direcciones de lectura/escritura**. BLP (confidencialidad) es **read down / write up**; Biba (integridad) es lo opuesto. No alcanza con saber “arriba/abajo”: hay que decir bien si es confidencialidad o integridad.

1.3. Modelos de Integridad: Biba y Clark-Wilson

1.3.1. Modelo Biba

Creado por Kenneth J. Biba. Fue el primer modelo público que **no buscaba confidencialidad sino proteger la integridad**. Es en realidad una familia de ~ 7 modelos. Define **niveles de integridad** (calidad/confiabilidad de la información). Una clasificación alternativa que el profesor prefiere: *información no confiable, poco confiable, confiable, muy confiable, fidedigna*. Importante en sistemas documentales / de evidencia: contabilidad histórica, registros financieros que hay que retener, sistemas judiciales, medios de prensa, declaraciones fiscales.

Biba estricto. Es Bell-LaPadula al revés:

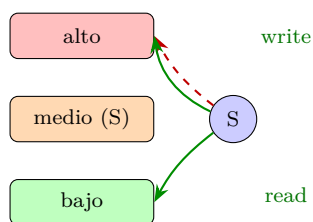
Definición: Reglas de Biba estricto

Lectura: un sujeto puede leer un objeto si y solo si el nivel de integridad del **objeto** es igual o superior al del sujeto $\Rightarrow$ “**no read down**”.

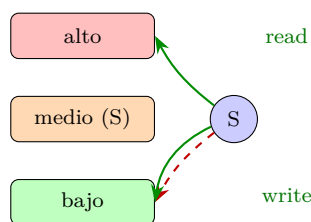
Escritura: un sujeto puede escribir un objeto si y solo si el nivel de integridad del **objeto** es igual o menor al del sujeto $\Rightarrow$ “**no write up**”.

Biba (integridad): READ UP, WRITE DOWN. (Invertir las flechas de un retículo BLP da Biba.)

BLP (confidencialidad)



Biba (integridad)



Biba es el espejo de BLP: las flechas de lectura y escritura están invertidas. Confidencialidad lee abajo / escribe arriba; integridad lee arriba / escribe abajo.

Intuición: “uno es tan confiable como las fuentes que utiliza”. Para considerar a un sujeto confiable, la información en la que se basa debe ser de igual o mayor confianza (no puede leer información dudosa). Y lo que un sujeto genera/escribe tiene a lo sumo su mismo nivel, nunca mayor. Ejemplo: el SO te advierte al ejecutar un archivo bajado de Internet (origen poco confiable) y no si lo bajaste del App Store.

Biba Low-Watermark. Variante **dinámica** (vs. las estáticas). No restringe la lectura: cualquier sujeto puede leer cualquier objeto, pero al hacerlo **el nivel de integridad del sujeto se reclasifica al mínimo** entre el que tenía y el del objeto. Si interactúa con algo de baja integridad, su nivel baja automáticamente, y deberá revalidarse para recuperar el nivel anterior. Base de muchos sistemas “adaptativos”.

► En el parcial

Biba Low-Watermark fue tomado en el Recuperatorio 2025. Recordá: *lectura libre, pero el sujeto baja su nivel al del objeto leído (toma el mínimo)*.

1.3.2. Modelo Clark-Wilson

Creado por David D. Clark y David R. Wilson. Modelo de **integridad** más acotado, pero introduce conceptos clave. Opera en un escenario con información y sujetos confiables y no confiables al mismo tiempo:

- Si sujeto e información son no confiables, que hagan lo que quieran.
- Un sujeto confiable no puede usar información no confiable; información confiable no puede ser usada por un sujeto no confiable.

¿Qué pasa cuando un sujeto confiable quiere usar información no confiable? Entra el mecanismo:

- Los datos pueden ser **restringidos/protegidos (CDI, Constrained Data Item)** o **sin restricción (UDI, Unconstrained Data Item)**.
- Los datos restringidos solo se modifican mediante un **Proceso de Transformación (TP)**.
- Hay un **Proceso de Verificación de Integridad (IVP)** independiente que valida.
- **Separation of duty (separación de tareas/privilegios)**: quien modifica los datos no puede ser el mismo que verifica la integridad.

Reglas de certificación y enforcement (del modelo formal): C1 IVP valida estado de CDI; C2 los TP preservan estado válido; C3 separación de tareas adecuada; C4 los TP escriben a log; C5 los TP validan los UDI. E1 los CDI se cambian solo por TP autorizados; E2 usuarios autorizados para TP; E3 usuarios autenticados; E4 las listas de autorización las cambia solo el security officer.

! Importante — remarcado en clase

“La idea de este modelo de arbitración de Clark-Wilson es la base por la cual existen los kernels del sistema operativo y existe el concepto de User Space / Kernel Space como cosas separadas.”

— dicho en clase Ejemplo bancario: para mover montos altos hacen falta dos personas (o procesos independientes), una verifica las firmas y otra ejecuta la transacción. En el SO, un *syscall* arma datos en user space, un proceso los transforma/mueve a kernel space, otro proceso verifica su integridad y recién ahí se ejecuta en el espacio protegido.

1.3.3. Más allá de los modelos teóricos

Son modelos teóricos, difíciles de aplicar enteros a un sistema. No esperes encontrar el nombre explícito, pero aparecen partes:

- Compartimentos de BLP → sistemas que manejan **información médica** (regulatorios).
- Biba → sistemas de clasificación y archivo para **medios de prensa, legajos judiciales, declaraciones fiscales**.
- Clark-Wilson → concepto **kernel space / user space**.

1.4. Control de Acceso: clasificación

Definición: Control de acceso

Mecanismo de seguridad que regula quién puede acceder a qué recursos de la aplicación y bajo qué condiciones.

Dos grandes familias:

Definición: Mandatory Access Control (MAC)

Control que se aplica en forma **obligatoria** a todos los accesos de los sujetos a los objetos. Existen reglas universales que se aplican a todos. Los modelos vistos (BLP, Biba) son MAC. Se aplica sobre: todos los accesos, los cambios en las reglas/atributos, y el intercambio de información entre sujetos.

Definición: Discretionary Access Control (DAC)

La asignación y modificación de permisos queda a **discreción** del sujeto responsable del objeto o de un administrador. Las propiedades de seguridad las da el administrador según cómo configure, no el sistema. Típico de repositorios donde los sujetos crean los objetos (file systems, object storage).

MAC es muy rígido; para sistemas de propósito general (SO, bases de datos, suites de oficina) los controles tienden a ser **discrecionales** por flexibilidad.

1.4.1. Control discrecional por extensión

Definir el conjunto de accesos *enumerándolos* uno por uno.

Matriz de acceso. Matriz centralizada: sujetos (filas) $\times$ objetos (columnas); en cada celda, las acciones permitidas para esa combinación. Simple de entender, pero con problemas: escalabilidad (crece violentamente), eficiencia de búsqueda, mantenimiento propenso a errores, espacio, y mala adaptación a entornos dinámicos. Para una computadora una matriz enorme no es problema; **el cuello de botella es el humano** que debe cargar los valores a discreción.

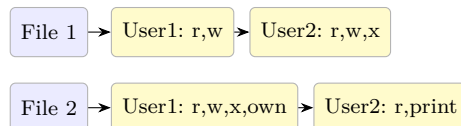
△ Cuidado — error típico

Falso clásico de parcial: “**una matriz de acceso bien diseñada evita el flujo de información no deseado**”. **FALSO**. La matriz (DAC, por extensión) solo controla el acceso *directo*; no garantiza control de flujo de información. Para eso hacen falta modelos MAC tipo Bell-LaPadula (la *-property es justamente lo que controla el flujo).

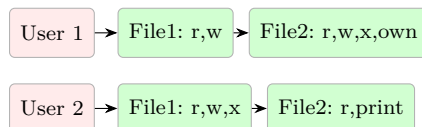
		Objetos→	
		File 1	File 2
Sujetos	User 1	r, w	r, w, x, own
	User 2	r, w, x	r, print

Matriz de acceso: sujetos (filas) $\times$ objetos (columnas); cada celda lista las acciones permitidas (basado en la slide “Control por extensión”).

ACL (por columna = objeto)



Capabilities (por fila = sujeto)



*Descomposición de la matriz: **ACL** recorre por columnas (a cada objeto, quién puede); **Capabilities** recorre por filas (a cada sujeto, a qué puede). Misma información, distinta forma de almacenarla.*

ACL (Access Control List). Como las matrices reales son **dispersas** (permisos por “islas”), se representan de forma compacta: para cada objeto, la lista de pares (sujeto, acción) permitidos. Es la misma información que la matriz, pero compacta. Se complementa con **permisos por defecto**, **grupos/agrupaciones** y **patrones** (/users/*, /home/**/*.*tmp).

1.4.2. Control discrecional por comprensión

Definir el conjunto de accesos por sus **propiedades**, mediante **reglas** más compactas que reconstruyen la matriz bajo demanda. Tres modelos según criterio de agrupación:

- **RBAC (por roles)**: se estereotipan **roles** (Account Manager, DBA, Gestor de pacientes, Analista de compras...) que agrupan permisos; los roles se asignan a sujetos o grupos. Tras definir los roles, la gestión diaria es simple. (En sistemas tradicionales un usuario tiene un solo rol; en los complejos puede tener varios, lo que genera conflictos.) Algunos libros lo llaman MAC; en realidad la asignación de roles suele ser discrecional.
- **ABAC (por atributos)**.
- **RuBAC (por reglas)**: no asigna sujetos como los roles; usa una **lista ordenada de reglas** (Principal, Objeto, Tipo de Acceso, Permitido/Bloqueado) y aplica una política de resolución.

△ Cuidado — error típico

Para usuarios **anónimos** conviene **ACL sobre Capabilities (CAP)** (parcial 2018). Las capabilities asocian permisos al sujeto (necesita identidad/token); con usuarios anónimos no hay a quién asociar la capability, así que conviene listar en el objeto quién puede (ACL).

1.5. Resolución de conflictos entre reglas

Cuando varias reglas aplican de forma contradictoria (o cuando se combinan varios mecanismos de control sobre la misma interacción), hay que resolver el conflicto. Es **crucial** entender cómo resuelve cada sistema, porque *el mismo juego de reglas da resultados distintos según la política de resolución*.

- **First match**: las reglas tienen orden; se aplica la **primera** que hace match.
- **Best match**: aplica la regla **más específica** (antes que las generales).
- **Deny over Allow**: cualquier regla que deniegue explícitamente tiene precedencia sobre las que permiten (equivale a: si hay múltiples reglas que aplican, *todas* deben permitir; si no, se deniega).
- **Grant-all / augment / override**: políticas adicionales (grant-all $\approx$ permitir si alguna regla permite).

Ejemplo (firewall). Reglas ordenadas; conexión $10,1,1,1 \Rightarrow 20,1,1,1:80$. Matchean la regla específica ($10.1.1.1 \dots \text{Accept}$) y la general ($10.1.1.0/24 \dots \text{Deny}$).

- **First match:** gana la primera $\Rightarrow$ **Accept**.
- **Best match:** gana la más específica $\Rightarrow$ depende del caso (para $10,3,3,9 \Rightarrow 20,3,3,9:80$, la específica **Deny** gana sobre $10.3.3.0/24$ **Accept**).
- **Deny over Allow:** como hay una regla que deniega, **se frena** $\Rightarrow$ **Deny**.

En first-match las reglas dicen “no permitas esta red salvo desde tal servidor”; en best-match “permití toda la red salvo tal IP”. La forma de escribir las reglas cambia según el motor.

► En el parcial

Ejercicio recurrente (2015, 2016, Recup 2023): dado $ACL(o) = \{(Manuel, r), (Directivos, w), (*, w)\}$, donde Manuel *es* Directivo, decidir bajo qué política puede **leer** y **escribir**.

- **Leer (r):** solo la regla $(Manuel, r)$ otorga lectura. Manuel **puede leer** bajo cualquier política que considere esa regla (first-rule si está primero, grant-all, augment).
- **Escribir (w):** las reglas $(Directivos, w)$ y $(*, w)$ otorgan escritura y Manuel encaja en ambas; ninguna regla la deniega, así que **puede escribir**.
- Bajo **grant-all**: alcanza con que una regla lo permita $\Rightarrow$ Manuel puede leer Y escribir.
- Razonar siempre: qué reglas matchean al sujeto, qué accede cada una, y cómo combina la política elegida (first-rule, grant-all, override, augment).

1.5.1. Reglas basadas en contexto

El **contexto** son los datos asociados al sujeto, objeto, momento o lugar del acceso, que cambian independientemente de las reglas: momento del día, ubicación geográfica del origen, departamento del que accede, si el usuario estaba de vacaciones, nivel de confiabilidad de la máquina. Permiten detectar anomalías:

- Login desde China y desde Argentina con 5 min de diferencia.
- Usuaría de licencia por maternidad que se loguea a una base de datos sensible.
- Usuario que abre dos VPN en menos de 5 min con dos IP distintas.

Su nicho son los **sistemas de prevención de fraude**. (Un fraude es un abuso *dentro* de las reglas del sistema; un problema de seguridad es un abuso *salteándose* las reglas.) Son complejos; en el modelo **Zero-Trust** se define una entidad externa (**Policy Engine**) que evalúa todas las condiciones de contexto.

1.5.2. Open Policy Agent (OPA)

Framework estándar que unifica, mediante un lenguaje programable, la implementación de políticas de decisión, **desacoplando** la política del sistema de control. El servicio delega cada decisión a OPA: le envía una *query* (JSON) y recibe una *decision* (JSON). OPA evalúa una **política (Rego)** contra **datos contextuales (JSON)**. Las políticas se escriben en el lenguaje **Rego** (ej. en Kubernetes, denegar despliegues de imágenes que no vengan de un dominio dado).

1.6. Ejemplos reales de control de acceso

- **Unix/Linux file system:** “engñosamente simple”. Permisos **rwX** en 3 secciones (usuario / grupo / otros). Tipo de archivo: **d** directorio, **-** archivo regular, **l** symlink. Al crear, un archivo hereda atributos del directorio; **umask** define una máscara que quita permisos a los archivos nuevos. Es **DAC**.
- **SELinux** (Security Enhanced): control **adicional al DAC**, basado en reglas que responden “¿puede este sujeto acceder a este objeto?”. Procesos y recursos tienen un *SELinux context* (ej. `/var/tmp: t_var_tmp`, `Port 80: t_port_www`). Usa macros que expanden a conjuntos de

reglas. Permite que dos procesos con UID 0 (root) tengan restringido el acceso a los recursos del otro. Se usa en *hardening* de servidores.

- **Windows file system:** DAC con dueño; usuarios y grupos; nivel básico y avanzado (más granular, ~ 11–13 operaciones). **Herencia** de permisos del contenedor, que *persiste a la creación* (a diferencia de Unix). Calcula **permisos efectivos**; **Deny tiene precedencia sobre Allow**; reglas locales sobre heredadas. Cada entrada puede Allow / Deny / dejar en blanco (en blanco = respeta el heredado).
- **PostgreSQL:** modelo basado en **roles** (un rol = usuario, grupo u otro rol; propiedad LOGIN). Todo objeto creado por un rol tiene a ese rol como **owner** (DAC). **Policies** (Row Level Security) permiten need-to-know a nivel de registros (ej. cada vendedor ve solo sus clientes vía `salesperson_id = current_user`).
- **Squid Web Proxy:** proxy web que controla por dónde navegan los usuarios. Sistema de reglas (mal llamado ACL) de tipo **MAC**; reglas ordenadas que filtran por dominio, protocolo, URL, headers, **listas negras (blacklists)**. En la facultad bloquea sitios de streaming con esta tecnología.
- **AWS:** combina dos mecanismos: **policies asociadas al Principal (MAC)** (definidas para la cuenta, por administradores) y **policies asociadas al Recurso (DAC)** (las define quien crea el recurso, ej. S3 bucket policy).
- **API Gateway:** capa delante de las APIs; control basado en reglas (MAC) por URL, dominio, headers, method; y por contexto (req/seg máximos, ancho de banda).

1.6.1. OAuth 2.0 (Open Authorization)

Estándar de **autorización** para que aplicaciones accedan a recursos en nombre del usuario **sin compartir sus credenciales**. Funciona emitiendo **tokens de acceso temporales y específicos**. Es el “entrar con tu cuenta de Google”.

Flujo: el usuario (Resource Owner) accede a la aplicación; la app lo redirige al **Authorization Server**, que lo autentica y le muestra la pantalla de consentimiento; al aceptar, el servidor envía un **authorization code** a la app, que lo intercambia por un **access token**; con ese token la app pide el recurso al **Resource Server**.

- **Access token** (vida corta: una hora, días) + **refresh token** (uso único; al usarlo devuelve un access y un refresh nuevos). Mecanismo para recuperarse ante robo de tokens y sostener la “sesión infinita”.
- **Scope (DAC):** permisos específicos que la app solicita (ver email, postear, leer calendario...). Se muestran en la pantalla de consentimiento; el token se limita a los scopes otorgados. Cada servicio verifica que el token sea válido y contenga su scope. Quien determina el control de acceso es **el dueño del recurso**.

1.7. Metodología: diseño de compartimentos (ejercicio estrella)

► En el parcial

Es el ejercicio: dados sujetos, objetos y requisitos en **lenguaje natural**, construir un retículo de compartimentos. Casos históricos:

- **Confidencialidad:** restaurante Michelin (parciales 2015 y 2016) ⇒ Bell-LaPadula.
- **Integridad:** clínica de cirugías estéticas (2013) y caso deepfake ⇒ Biba.

SIEMPRE hay que justificar requisito por requisito y nombrar bien la política.

Pasos.

1. **Identificar el objetivo de seguridad:** ¿el enunciado pide *ocultar* información (confidencialidad ⇒ BLP) o *proteger la calidad/origen* de la información (integridad ⇒ Biba / Clark-Wilson)? El deepfake/integridad: “Necesitan estructurarse con un modelo similar a Biba”.

2. **Listar sujetos y objetos** del enunciado.
3. **Definir los niveles** (jerarquía vertical) y las **etiquetas/categorías** (need-to-know horizontal: áreas, proyectos, temas que deben ir separados).
4. **Asignar a cada sujeto y objeto su compartimento** (nivel, {etiquetas}), justificando con el requisito que lo motiva.
5. **Dibujar el retículo** con la relación de dominancia (orden parcial): aristas hacia el que domina; compartimentos sin etiquetas comparables quedan separados.
6. **Verificar cada requisito** contra las reglas del modelo (read down/write up para BLP; read up/write down para Biba), uno por uno.
7. **Nombrar la política** explícitamente (BLP “no read up, no write down” / Biba “no read down, no write up”).

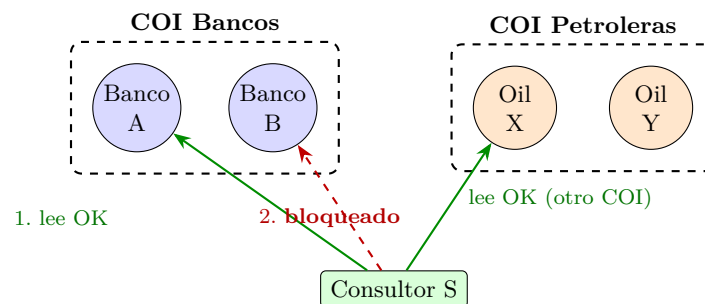
► En el parcial

Identificar la política dado un retículo (parcial 2018, jerarquía CEO → Gerentes → Empleado):

- Mirar las flechas/dirección de los flujos permitidos. Si el flujo de *lectura* va hacia abajo (un superior lee lo de abajo) y la *escritura* hacia arriba ⇒ **Bell-LaPadula** (confidencialidad).
- Si están **invertidas** (lectura hacia arriba, escritura hacia abajo) ⇒ **Biba** (integridad). Invertir las flechas de un retículo BLP da Biba.

1.7.1. Chinese Wall / Muralla China

Modelo de **conflicto de interés**. Se agrupan los objetos en **clases de conflicto de interés (COI / Conflict of Interest)**; dentro de cada clase hay **datasets de compañías (CD)**. La regla: un sujeto que accedió a datos de una compañía dentro de una clase de COI **no puede acceder a los datos de otra compañía competidora** de la misma clase (ya consultó a un competidor). Tomado en el parcial 2018.



*Chinese Wall: tras leer Banco A, el consultor **no puede** leer a Banco B (competidor del mismo COI), pero sí una petrolera de otro COI.*

1.8. Mentalidad de seguridad (clase de consulta)

! Importante — remarcado en clase

De la clase de consulta, el profesor remarcó:

- Los ejercicios son **situaciones laborales** donde uno *decanta* la política general a partir del enunciado.
- Hay que “**activar el chip paranoico**”.
- **Defensa en profundidad = encadenar controles** (varias capas).
- Para el caso deepfake/integridad: “*Necesitan estructurarse con un modelo similar a Biba*”.

► En el parcial

Ejercicio abierto tipo “sos el CSO”: pasos para responder.

1. **Identificar el objetivo de seguridad:** confidencialidad, integridad o disponibilidad (puede haber más de uno).
2. **Elegir el modelo** acorde (BLP para confidencialidad; Biba/Clark-Wilson para integridad).
3. **Definir la política** concreta (qué estados son seguros).
4. **Bajar a reglas** (ACL, RuBAC, compartimentos, MAC/DAC, resolución de conflictos) y, si conviene, encadenar controles (defensa en profundidad).

2. Autenticación

2.1. Qué es autenticar

Definición: Autenticación

Asociar una **identidad** a un **principal**. La identidad corresponde a una entidad externa (la entidad del entorno), y el principal es la representación interna del sistema (p. ej. “Usuario1”, “Sistema1”). El componente que realiza la asociación es el **autenticador**.

La entidad externa es generalmente física y debe representarse de alguna forma dentro del sistema. Esa representación interna se llama *principal* (típicamente el ID de usuario o algún identificador). Cada vez que un sujeto quiere autenticarse hay que comprobar que sea quien dice ser y no alguien que lo **impersona**.

Pasos de la confirmación de identidad.

1. Obtener información de identidad.
2. Analizarla (ver si es fidedigna).
3. Determinar si corresponde a una entidad del sistema.

Consecuencias: hay que almacenar información de cada entidad y contar con mecanismos para procesar esa información.

2.1.1. Componentes de un sistema de autenticación

Análogo a un criptosistema, es una tupla de **5 componentes**:

Comp.	Nombre	Descripción
$A = \{a\}$	Información de autenticación	Designación de la entidad e información provista. La provee la entidad externa; el sistema <i>no</i> la controla.
$C = \{c\}$	Información complementaria	Especificación de un principal + información almacenada por el sistema.
$F = \{f : A \rightarrow C\}$	Funciones de complementación	Derivan la información complementaria a partir de A .
$L = \{l : A \times C \rightarrow \{0, 1\}\}$	Funciones de autenticación	Determinan si un par (A, C) es una asociación válida.
$S = \{s\}$	Funciones de selección	Permiten crear y actualizar A y C (administrativas).

! Importante — remarcado en clase

Distinción F vs L (el profe la aclaró por pedido de un alumno):

- F *deriva* C a partir de la información de autenticación A (ej: $C = h(A)$, el hash de la clave).
- L *corroborar*: dada una *nueva* información de autenticación, dice si existe una C correspondiente almacenada en el sistema.

S son administrativas (crear/actualizar/borrar entidades), p. ej. `adduser()`, `removeuser()`, `passwd()`.

Es deseable que **de C no se pueda volver a A** , porque si tengo C podría saber a quién pertenece la información (o incluso ingresar al sistema).

► En el parcial

Función de complementación (2016). Dada información de autenticación e información complementaria, retorna V/F. *Cuidado:* eso describe a la función de **autenticación** L ($A \times C \rightarrow \{0, 1\}$). La función de **complementación** F es $A \rightarrow C$ (deriva C). Tener clara la diferencia.

► En el parcial

Módulos de Autenticación vs Control de Acceso (2013). Diagrama:

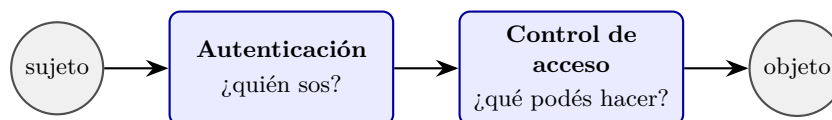
sujeto $\rightarrow$ módulo autenticación $\rightarrow$ módulo control de acceso $\rightarrow$ objeto

Piden dar una vulnerabilidad de *cada* módulo. Ideas del material:

- **Autenticación:** robo/adivinación de la clave (fuerza bruta, diccionario), impersonación, robo del segundo factor, robo de la base de información complementaria.
- **Control de acceso:** una vez impersonado un usuario, el sistema rara vez detecta que “ese no sos vos”; fallas en las reglas que definen si un sujeto accede a un objeto y de qué forma (MAC/DAC).

“Una vez que te personaste como otro usuario, ya está; el sistema, si no tiene métodos fuertes de detectar que ese no sos vos, por lo general no lo tiene, y podés hacer lo que quieras dentro del sistema.”

— dicho en clase



Dos módulos distintos: primero se verifica la identidad (autenticación), luego qué operaciones se permiten sobre el objeto (control de acceso). Cada módulo tiene sus propias vulnerabilidades.

2.2. Factores de autenticación

La información de autenticación posee distintos **grados de confianza**. Los *factores* indican la naturaleza de la información.

Factor	Descripción y ejemplos
Algo que conozco	Secreto compartido. Clave, frase (passphrase), pregunta secreta. Requiere almacenar de forma segura el secreto; depende de su confidencialidad. Es el mecanismo más empleado y base de otros.
Algo que tengo	Elemento físico. Smart card / USB token, generador de claves (RSA-ID), celular, tarjeta de coordenadas (token grid). Requiere cuidar el elemento; puede copiarse/clonarse; la info en tránsito puede ser suplantada.
Algo que soy	Características intrínsecas (biometría). Huella, retina, voz, cara. No fácilmente modificables. Métodos no 100 % eficaces; asumen que el dispositivo de lectura no puede manipularse.
Contexto	Red de origen, país/ciudad/geolocalización, host/terminal/device, día y hora, velocity. Suele usarse como complemento, no como único factor. Reglas positivas (operador desde la red privada del datacenter) o negativas (acceso desde otro país ⇒ menos chances de ser quien dice).



Los tres factores clásicos de autenticación. A más factores independientes combinados, mayor grado de confianza.

! Importante — remarcado en clase

La información de autenticación la controla la entidad externa; el sistema no puede “ingerirla”/forzarla, pero sí puede **exigir ciertos factores** que gradúan la confianza. No es lo mismo que alguien diga una contraseña (que pudo haber escuchado) a que muestre su cara: distintos grados de confianza.

2.2.1. Biometría

No es 100 % eficaz por dos motivos: (1) no hay sistema de lectura siempre eficaz, y (2) aunque lo hubiera, la representación cambia (cambios físicos, etc.). Las primeras implementaciones de huella en celulares rechazaban al usuario; muchos sistemas **subieron el umbral de error** para no bloquear, aceptando lecturas imperfectas. Requiere un **mecanismo adicional** para evitar engaños (foto al lector de cara, manipular el dispositivo de lectura): se ponen cámaras u otros mecanismos para que no se manipule el lector.

! Importante — remarcado en clase

Puede pasar que la **misma función complementaria** valide *dos* informaciones de autenticación distintas: el caso real es el celular de una persona que se desbloquea con la cara de un hermano/primo parecido (falso positivo biométrico).

► En el parcial

Robo de identidad biométrico (V/F clásico, 2013). Verdadero: sí puede haber robo de identidad biométrico. Los datos biométricos pueden ser capturados/suplantados, el dispositivo de lectura manipulado, y existen falsos positivos (rasgos parecidos). La biometría no es infalible.

2.3. Claves (passwords)

A diferencia de las claves criptográficas, las passwords **las controla el usuario** y por lo general *no* son aleatorias. El sistema sí puede tomar decisiones sobre qué permite como clave.

Tipos.

- **Secuencias de caracteres:** con restricciones (largo, dígitos, mayúsculas, símbolos); generadas aleatoriamente, por el usuario o de manera asistida.
- **Secuencias de palabras (pass-phrases):** más largas. *Cuidado:* a veces son **más adivinables** que una contraseña normal (ej. “perro árbol 123”).
- **Algorítmicas:** challenge-response (pregunta-respuesta) y OTP (claves de uso único).

2.3.1. Almacenamiento de claves

Método	Comentario
Texto en claro	NO recomendado. Si roban la BD se llevan todo; accesible por fuera del sistema; los administradores/desarrolladores con acceso a la base podrían ver los secretos.
Archivo cifrado	Requiere una clave para acceder. La clave puede estar en config, en el ejecutable, ingresarse al iniciar, o en un dispositivo criptográfico. Solo recomendado si <i>se necesita</i> la clave original (p. ej. reenviarla a un sistema legacy).
Hash / func. de derivación	Forma correcta: guardar una <i>prueba</i> de que se conoce el secreto, sin guardar el secreto. Más abajo: salt + PBKDF2.

! Importante — remarcado en clase

Idea central del almacenamiento moderno: “*Se empieza a guardar como una prueba de que yo conozco el secreto, pero no guardo el secreto.*”

— dicho en clase La clave del usuario **nunca se almacena**; se valida sin guardarla.

Ejemplo: Unix legacy. Se almacena por cada usuario el resultado de **una de 4096** funciones de hash, elegida aleatoriamente.

- $A = \{\text{secuencias de hasta 8 caracteres}\}$
- $C = \{xx \underbrace{HHHHHHHHHHH}_{11}\}$, donde xx = función de hash (0–4095, 2 caracteres) y $HH...H$ = resultado (11 caracteres).
- $F = \{DES_{xx}(\cdot)\}$ (variante de DES, *no* reversible).

- $L = \{\text{login, su, sudo}\}; S = \{\text{passwd, adduser}\}.$

Marcó un precedente: hasta entonces no había consenso de cómo guardar contraseñas (se cifraban). Unix logró generar información complementaria de la que *no se puede volver* a la original.

2.3.2. Dispositivo criptográfico (HSM) y token físico

Las claves pueden guardarse en un **Hardware Security Module (HSM)** / cripto-chip especializado. Se usa cuando se requiere acceder a la clave original (contextos PCI, tarjetas de crédito, sistemas legacy de bancos que exigen la contraseña original).

► En el parcial

Token / llave física como segundo factor. Sirve porque la **clave privada no se puede extraer del hardware** (HSM/cripto-chip): “la clave nunca sale de ahí”. En los celulares hay un chip dedicado a biometría, por fuera del kernel del SO; el kernel solo recibe **OK/fail**, no accede a los datos biométricos.

2.4. Ataques a un sistema de autenticación

Objetivo del atacante: ser identificado como una entidad (impersonarla). **Mecanismo:** encontrar $a \in A$ tal que, para algún $f \in F$, $f(a) = c$ y c está asociado a una entidad.

Tipo	Descripción
Offline	Se conocen f y c ; se prueban distintos a hasta que $f(a) = c$. Ataca la función <i>complementaria</i> . Ej: obtener <code>/etc/shadow</code> (Linux) y usar <code>crack / john the ripper</code> ; obtener el archivo SAM (Windows) con <code>ophcrack</code> .
Online	Se usa la función L , probando distintos a hasta pasar la autenticación. Ataca la función de <i>autenticación</i> (el login). Ej: probar la función de login con una cuenta conocida (root, administrator, guest), o atacar el endpoint con herramientas tipo Postman/inspector.

Mejores ataques (no fuerza bruta pura). Asumiendo que las claves no son aleatorias:

- Diccionarios de palabras; diccionarios de claves *descubiertas* (leaks/breaches históricos).
- Transformaciones simples: sufijos, prefijos, reemplazos ($l \rightarrow 1, o \rightarrow 0$, vocales por números), palabras espejadas, dos palabras combinadas.
- Información de contexto: nombre, usuario, DNI, fecha de nacimiento (la peor de todas: el nombre de usuario).

2.4.1. Prevenciones

General (offline): esconder información para que a, c o f no sean conocidas simultáneamente. Como f suele conocerse (algoritmos estándar, reversing), es más fácil proteger c (ej. `/etc/shadow` solo accesible por root) que a (en manos del usuario).

Prevenir el uso de L (online):

- Tiempos crecientes ante fallas (**exponential back-off**): tras fallar se aumenta el tiempo de reintento exponencialmente.
- Deshabilitar principales que están siendo forzados (*cuidado*: un malicioso puede afectar la **disponibilidad** de un usuario haciéndolo bloquear).

- Jailing / Honeypot: al detectar ataque, se switchea L por una falsa; se deja al atacante en un ambiente controlado mientras se lo rastrea (“bote de miel”).
- Captchas (cada vez más vulnerados por bots e IA; baratos, ofrecidos como servicio).

Otras: revisión proactiva de claves (rechazar claves fáciles, buscar en diccionarios/leaks, detectar patrones, usar info del usuario), expiración/rotación de claves (limita el daño de una clave comprometida; evitar reuso de las últimas N ; *no* forzar cambio al momento de login; avisar con anticipación), subir la complejidad de las claves.

△ Cuidado — error típico

La **rotación de contraseñas** mal hecha es peligrosa: si se fuerza el cambio justo al ingresar (sin preaviso), el usuario apurado elige una contraseña fácil y **perjudica** la seguridad en vez de ayudarla. Siempre dar preaviso (7, 5, 1 día).

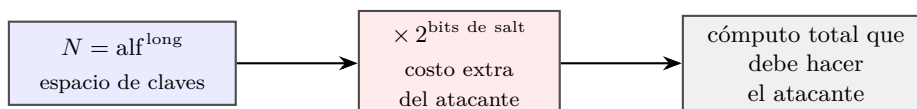
2.5. Fuerza bruta: fórmula de Anderson y metodología

Definición: Fórmula de Anderson

$$P = \frac{T G}{N}$$

donde P = probabilidad de adivinar una clave, G = número de pruebas realizables por segundo, T = tiempo dedicado a las pruebas, N = tamaño del espacio de claves. Sirve para estimar el tiempo de un ataque.

El **espacio de claves** es $N = (\text{tamaño del alfabeto})^{\text{longitud}}$.



El espacio de claves crece como $\text{alfabeto}^{\text{longitud}}$. El salt no agranda ese espacio, pero multiplica el cómputo del atacante offline por $2^{\text{bits de salt}}$ (no puede paralelizar entre cuentas).

► En el parcial

EL ejercicio que “entra siempre”. El profe lo dijo: es fácil de hacer, fácil de chequear y permite “darle vueltas de rosca”. Es la **segunda unidad más recurrente**.

Metodología paso a paso:

1. Identificar las variables: alfabeto ($\Rightarrow N = \text{alf}^L$), G (pruebas/seg), T (tiempo), P (probabilidad objetivo) y la incógnita.
2. Plantear $P \geq \frac{T G}{N}$ y despejar la incógnita.
3. **Pasar todo a la misma unidad.** Tiempo: 1 año = $365 \cdot 24 \cdot 60 \cdot 60$ s. Probabilidad objetivo típica 0,5 o 1/100.
4. Si la incógnita es la longitud L , despejar N y luego $L \geq \log_{\text{alf}}(N)$, redondeando **hacia arriba**.

Ejemplo resuelto (PDF, alfanuméricas 36 caracteres). ¿Qué longitud mínima reque-

rir para que $P < 0,5$ de hallar la clave en menos de 1 año, con $G = 100,000$ claves/seg?

$$P \geq \frac{T G}{N} \Rightarrow N \geq \frac{T G}{P} = \frac{(365 \cdot 24 \cdot 60 \cdot 60) \cdot 100,000}{0,5} \quad (1)$$

$$N = 36^L \geq 6,3 \cdot 10^{12} \quad (2)$$

$$L \geq \log_{36}(6,3 \cdot 10^{12}) \approx 8,22 \Rightarrow L \geq 9 \quad (3)$$

Otros ejemplos del PDF (despejando T):

- Numéricas de 10 dígitos ($N = 10^{10}$), $G = 10,000/\text{seg}$, $P > 0,5$: $T \leq \frac{P N}{G} = \frac{0,5 \cdot 10^{10}}{10,000} = 500,000 \text{ s} \approx 6 \text{ días}$.
- 8 letras (26 caracteres, $N = 26^8$), $G = 10,000/\text{seg}$, $P > 0,5$: $T \leq \frac{0,5 \cdot 26^8}{10,000} \approx 1,044,135 \text{ s} \approx 121 \text{ días}$.

► En el parcial

Variantes reales de parcial:

- Alfabeto $[a-zA-Z0-9]$ (62 símbolos), 10^4 passwords/seg: ¿5 símbolos y 5 bits de salt dan $P < 0,5$ en un año? (2015, 2025-1Q).
- Alfabeto 96 caracteres, $10^5/\text{seg}$, $P = 1/100$ en 365 días: hallar longitud mínima (Recup 2025).
- TPU con 700 claves/seg, clave de 12 bits: tiempo para $P = 1/100$ (Recup 2023). Con clave de 12 bits, $N = 2^{12}$.

Cuando la clave se da **en bits**, $N = 2^{\text{bits}}$. Conviene trabajar con potencias de 2 o logaritmos.

△ Cuidado — error típico

Errores que penalizan en el ejercicio de fuerza bruta:

- No pasar todo a la **misma unidad** (mezclar bits vs. símbolos vs. tiempo). 1 año = 365 días = $365 \cdot 24 \cdot 60 \cdot 60 \text{ s}$.
- Olvidar redondear L **hacia arriba** (entero).
- Invertir mal la desigualdad al despejar.

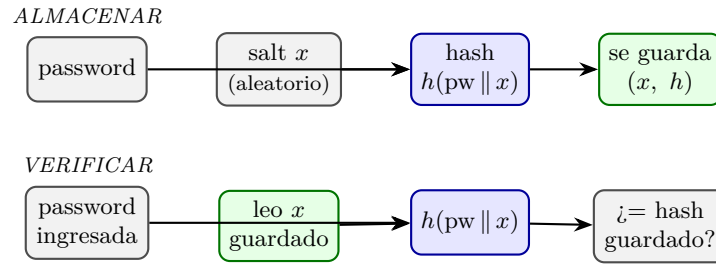
2.6. Salt (salting)

Escenario: un atacante consigue $c_1, c_2, \dots, c_n$ (una base robada de información complementaria). Sin salt, cada prueba $a_i \rightarrow f(a_i)$ puede compararse *en paralelo* contra todos los c_j .

Método: introducir una perturbación distinta por cuenta:

$$f(a) = x \parallel f'(a, x)$$

donde x es el salt (elegido aleatoriamente y embebido en el hash). Cada c se calcula con una perturbación diferente, por lo que el atacante **no puede reutilizar** $f(a)$ para buscar varias cuentas en paralelo, y debe obtener el salt de cada usuario.



La contraseña nunca se guarda: se almacena (salt, hash). Al verificar se recalcula el hash con el mismo salt y se compara. El salt hace que dos passwords iguales den hashes distintos.

! Importante — remarcado en clase

Qué hace y qué NO hace el salt (aclaración del profe):

- **NO** agranda el espacio de claves del usuario.
- **SÍ** penaliza al atacante *offline*: equivale a sumar bits al cómputo del atacante, aumentando su costo por $2^{\text{bits de salt}}$, porque tiene todos los hashes y debe probar más (no puede paralelizar ni detectar patrones de claves repetidas).
- Evita que dos contraseñas iguales den la misma información complementaria.

△ Cuidado — error típico

Error típico: decir que el salt **aumenta el espacio de búsqueda/de claves del usuario**. **Falso**. El salt no cambia el espacio de claves del usuario; penaliza al atacante (multiplica su costo por $2^{\text{bits de salt}}$). Por eso en el ejercicio “5 símbolos + 5 bits de salt” el salt suma al *cómputo del atacante*, no al alfabeto/longitud de la clave.

2.6.1. Funciones lentas: PBKDF2 (PKCS 5 v2.0)

Idea: encarecer el cómputo para que ejecutar el algoritmo sea costoso (suelen tardar ≥ 100 ms en vez de 10 ms), de modo que probar muchas passwords sea lento para el atacante, pero irrelevante para el sistema (lo corre una sola vez al validar).

Sea *pass* una contraseña, *S* un salt y PRF una función pseudoaleatoria (criptosistema simétrico o MAC). Se aplica iterativamente:

$$u_1 = \text{PRF}(pass, S) \quad (4)$$

$$u_2 = \text{PRF}(pass, u_1) \quad (5)$$

$$\vdots \quad (6)$$

$$u_j = \text{PRF}(pass, u_{j-1}) \quad (7)$$

$$\text{Resultado} = u_1 \oplus u_2 \oplus \dots \oplus u_j \quad (8)$$

Algoritmo completo: $K = \text{PBKDF2}(prf, pass, salt, c, len)$, con $K = T_1 \| T_2 \| \dots \| T_n$ ($n = len/|prf|$) y cada bloque $T_i = u_1 \oplus u_2 \oplus \dots \oplus u_c$, donde

$$u_1 = \text{PRF}(Pass, Salt \| \text{INT32_BE}(i)), \quad u_k = \text{PRF}(Pass, u_{k-1}).$$

! Importante — remarcado en clase

El parámetro c (cantidad de **rondas**/iteraciones) es ajustable: a más rondas, más caro para el atacante. Como el sistema solo lo calcula una vez (al guardar o al verificar), no importa cuánto tarde; al atacante sí le complica las cosas. El XOR final busca uniformizar los bits (sin patrones detectables). Ya está implementado en estándar: en Java, JCE provee PBKDF2WithHmacSHA1 vía `SecretKeyFactory` (no hay que implementar nada).

► En el parcial

¿Es válido guardar passwords con SHA-256 en una BD? (Recup 2025). Discutir:

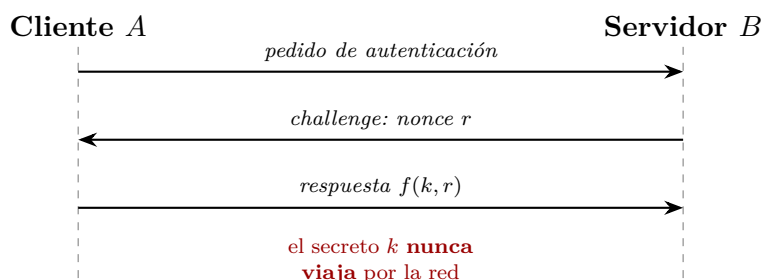
- Hashear es mejor que texto plano (guardás una prueba, no el secreto).
- Pero SHA-256 “a secas” es **rápido** $\Rightarrow$ vulnerable a fuerza bruta/diccionario/rainbow tables.
- Falta **salt**: sin él, dos claves iguales dan el mismo hash y el ataque se paraleliza.
- Lo correcto es una **función lenta** con salt y rondas configurables (PBKDF2, bcrypt, scrypt). Conclusión: SHA-256 solo *no* es adecuado.

2.7. Challenge-Response y EKE

Problema: autenticar a un usuario remoto requeriría enviar la clave (necesita canal seguro; descubrir una comunicación antigua puede comprometer la clave).

Solución (challenge-response): no enviar la clave.

1. $A \rightarrow B$: pedido de autenticación.
2. $B \rightarrow A$: un challenge $\{r\}$.
3. $A \rightarrow B$: $f(k, r)$ (aplica una función preacordada sobre su clave k y el challenge r).



Challenge-response: el servidor manda un desafío único r (nonce); el cliente responde $f(k, r)$ probando que conoce el secreto k sin enviarlo. El nonce evita reutilizar respuestas capturadas.

Así el usuario valida que conoce el secreto sin enviarlo. (Las *passkeys* actuales siguen esta idea: el servidor manda un challenge, el cliente lo firma con una clave protegida por biometría/PIN, y el servidor verifica; hay un esquema de clave pública/privada de fondo y no se envía ningún dato secreto.)

EKE (Encrypted Key Exchange). Objetivo: impedir que un atacante adivine claves capturadas en la red. Mecanismo: hacer el diálogo en un **canal encriptado** (con clave k_s), de modo que el atacante **no tiene forma de saber si una clave es correcta**.

$$A \rightarrow B : \{\text{pedido}\}_{k_s} \quad B \rightarrow A : \{r\}_{k_s} \quad A \rightarrow B : \{f(k, r)\}_{k_s}$$

2.8. OTP — Claves de uso único

△ Cuidado — error típico

No confundir OTP (One Time Password) con One Time Pad. Misma sigla, nada que ver.

Objetivo: generar claves que sirven una única vez. A y B comparten una clave K y un contador C . Concepto: $(K, C) \rightarrow \text{OTP} \rightarrow (otp, C')$. La clave K es criptográfica, generada aleatoriamente (es lo que se escanea en el QR / se carga en el authenticator). Dos variantes estandarizadas:

2.8.1. HOTP — RFC 4226 (basado en HMAC)

$$\text{HOTP}(K, C) = \text{Truncate}(\text{HMAC-SHA1}(K, C))$$

Truncate extrae 32 bits de los 160 del MAC:

- $offset = mac[19] \& 0xf$ ($\leftarrow$ 4 bits del último byte)
- $bin = mac[offset..offset+3] \& 0x7fffffff$
- $Result = bin \% 10^n$ (n = cantidad de dígitos)

El **contador se incrementa en cada uso** $\Rightarrow$ requiere sincronización (**look-ahead**: el servidor calcula varios intentos hacia adelante). Se aconseja **throttling**. Genera códigos de 1 a 9 dígitos (recomendado ≥ 6).

2.8.2. TOTP — RFC 6238 (basado en tiempo)

$$\text{TOTP}(K) = \text{HOTP}(K, T), \quad T = \frac{(\text{current unix time} - T_0)}{X}$$

donde T_0 = tiempo de referencia (usualmente 0) y X = segundos de vida de cada código (usualmente 30). **No requiere contador** (el “contador” lo da el tiempo). Se aconseja un **drift-window** (cantidad de ticks desfasados aceptados, hacia atrás y hacia adelante, por des-sincronización entre servidores) y **throttling**. Es el más usado hoy (Google Authenticator, etc.).

! Importante — remarcado en clase

La complejidad frente a fuerza bruta es la misma aunque el código rote: si son 6 dígitos, se puede probar por fuerza bruta igual. Por eso se necesita throttling del lado del servidor. Los **recovery codes** son códigos de un solo uso para recuperar la cuenta si se pierde el dispositivo del segundo factor.

2.9. Multifactor y multicanal

Multi factor authentication: requerir dos (o más) factores de *distinto tipo*. Racional: cada tipo de factor requiere comprometer distintos *assets*. Combinaciones comunes: conozco+tengo, conozco+soy, tengo+soy.

► En el parcial

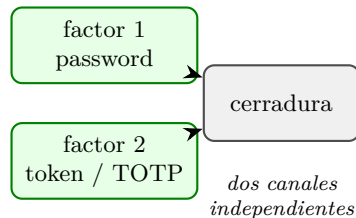
Independencia de factores (concepto clave). El multifactor es más seguro porque obliga a comprometer **varios canales independientes**. PERO si los factores están **entrelazados**, se anula la independencia y pierde sentido.

- Ejemplo del profe: el **celular** concentra “algo que tengo” + “algo que soy” (ca-

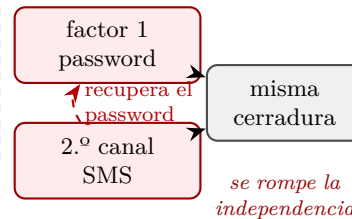
ra/huella) en un solo dispositivo. Si lo roban/clonan/comprometen, comprometen *ambos* factores a la vez.

- Ejemplo de entrelazamiento que rompe la independencia: poder **recuperar el password por el segundo canal** (SMS) $\Rightarrow$ “dos llaves para la misma cerradura”, ya no hay independencia real.

MULTIFACTOR (ok)



MULTICANAL entrelazado



Izq.: dos factores independientes, hay que comprometer dos canales distintos. Der.: si el 2.º canal (SMS) sirve para recuperar el password, ambos factores quedan entrelazados —“dos llaves para la misma cerradura”— y se pierde la independencia.

△ Cuidado — error típico

Error típico: decir que multifactor “es seguro” sin notar que **factores entrelazados rompen la independencia**. Hay que señalar la dependencia (segundo factor recuperable por el primero, ambos en el mismo dispositivo, etc.).

! Importante — remarcado en clase

Trilema usabilidad / performance / seguridad. “Seguridad siempre es un balance, algo vas a penalizar.”

— dicho en clase Pedir el segundo factor todo el tiempo es seguro pero golpea la usabilidad; por eso existe la **step-up authentication**: pedir un factor adicional solo ante anomalías (contexto extraño, cambio de geolocalización, primer login en mucho tiempo, etc.).

2.10. Autenticación remota (SSO / Single Sign-On)

Consiste en **delegar la autenticación** a un sistema externo (relación de confianza). Conviene cuando otra empresa lo hace de forma más robusta y uno quiere enfocarse en su negocio.

- **Productos propietarios:** Active Directory Federation Services (ADFS), CAS, Siteminder. (El ITBA usa Active Directory: los usuarios viven en el AD como recursos.)
- **Tecnologías abiertas:** OpenID 1 y 2 $\rightarrow$ obsoletos. **OpenID Connect** $\rightarrow$ basado en OAuth2.

! Importante — remarcado en clase

OpenID Connect vs OAuth2. OAuth2 es **autorización**; OpenID Connect le **añade la capa de autenticación** encima de OAuth2. “Login con Google/Facebook/Apple” delega la autenticación: la parte de saber *quién sos* (nombre, foto) es OpenID Connect; pedir acceso

a tus recursos (drive, archivos) es OAuth2. Así no hay que crear un usuario nuevo en cada sistema externo (federated login).

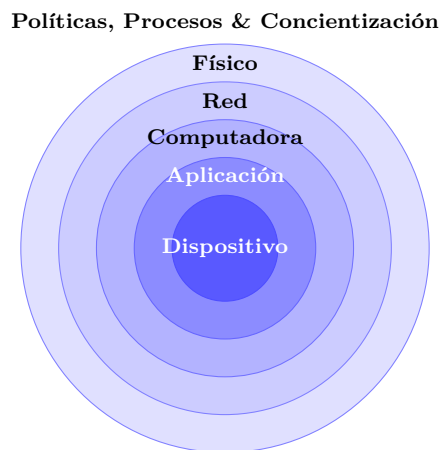
2.11. Lectura recomendada

Capítulos 12–13 de *Computer Security: Art and Science*, Matt Bishop.

3. Principios de Diseño de Seguridad y Vulnerabilidades

3.1. Introducción: qué son los principios de diseño

Los **principios de diseño** son bases que promueven un diseño que tenga como resultado un sistema **robusto y seguro**. Son principios guía de *alto nivel* que deben adaptarse a las necesidades puntuales de cada sistema, se basan en las recomendaciones de los modelos de seguridad y deben aplicarse en **todas las capas** en donde funciona el sistema (políticas/procesos, físico, red, computadora, aplicación, dispositivo). Son simples y restrictivos.



Los principios se aplican en todas las capas en las que vive el sistema, del perímetro (políticas/procesos) hacia el núcleo (el dato/dispositivo).

! Importante — remarcado en clase

Que un sistema sea **robusto** significa que pueda aguantar cambios. El primer requisito de seguridad es que el sistema **esté disponible**: un ataque que tira el sistema (DoS) *es* un ataque de seguridad, aunque no comprometa confidencialidad ni integridad.

Todo aspecto de seguridad involucra tres patas que conviven (no alcanza con sola la tecnología):

- **Tecnología**: las herramientas (lo que más se ve en esta materia).
- **Procesos**: los procesos de las compañías; en sus grietas se mete la inseguridad.
- **Personas**: la pata psicológica / ingeniería social. *“Es el punto más crítico por donde el sistema cae.”*
— dicho en clase

! Importante — remarcado en clase

La seguridad es un **balance entre tres cosas**: seguridad, funcionalidad y eficiencia/performance (“bueno, bonito y barato”). Un sistema muy seguro suele ser difícilísimo de usar. La seguridad siempre es tan fuerte como el **eslabón más débil** de la cadena.

3.2. Principios de Saltzer & Schroeder (1975)

Propuestos en **1975** para el diseño de *sistemas operativos seguros*. Demostraron aplicabilidad en diseños generales y sistemas modernos. Son la base de buenas prácticas modernas y estrategias de gobierno en todas las capas. Muchos vienen del escenario militar.

! Importante — remarcado en clase

El profe remarcó que estos principios “hay que grabarlos afuera porque son los más importantes”. Aparecen todo el tiempo de una u otra manera y **están todos conectados / se tocan entre sí**. El objetivo de la materia es generar la **intuición** para aplicarlos.

3.2.1. 1. Mínimo privilegio (Least privilege)

Se debe asegurar que los sujetos (personas, entidades, procesos) sólo tengan el **acceso necesario** (la menor cantidad de permisos/capacidades) para realizar su trabajo, y nada más.

- Ejemplo: en un sistema con información financiera confidencial, limitar quién accede. La persona que atiende el teléfono no necesita acceder a todo; el administrador de cuentas sí, pero *no* a las cuentas que no administra.
- Por detrás surge que la **condición por default es no tener acceso a nada**.

Definición: Mínimo privilegio

Dar a cada sujeto únicamente los permisos/capacidades mínimas necesarias para realizar la tarea que tiene que hacer, y ningún permiso adicional. Por default no se tiene acceso a nada.

3.2.2. 2. Fallar de forma segura (Fail-safe defaults)

Conectado al anterior: el default es “no dejo entrar a nadie”.

- A la hora de revisar, las **listas de permitidos (whitelists)** son mejores que las listas de excluidos (**blacklists**). Conviene marcar quién *puede* entrar, no quién no puede. (Ejemplo del profe: Twitter pasó de white-listing a black-listing por razones económicas/de influencia, pero en general el white-listing es mejor.)
- Un sistema no abre puertos innecesariamente; por default los firewalls están encendidos.
- No hay cuentas de usuario por defecto con contraseña conocida.
- Un sistema que presenta una falla en un proceso o control de seguridad debe moverse a un **estado seguro**.
- Ejemplo: si el sistema de autenticación no puede conectarse a la base de datos para verificar un password, debe **rechazar el pedido** y no asignar permisos, por más que sean los mínimos. Lo que se penaliza al fallar seguro es *funcionalidad*.

Ejemplo extremo de hardware: las normas FIPS de dispositivos físicos (HSM, criptochips/tokens) tienen niveles (1 a 4); en nivel 4, ante una amenaza física el dispositivo debe desaparecer o no debe haber forma de extraer la clave privada (anti-tampering).

3.2.3. 3. Simplicidad (Economy of mechanism)

La complejidad es enemiga de la seguridad. Se debe evitar arquitecturas muy sofisticadas al desarrollar controles de seguridad.

- Las soluciones complejas **exponen más superficie de ataque**, son menos robustas y más difíciles de entender; es más fácil que se cuelen bugs que deriven en vulnerabilidades explotables.
- Busca reducir la superficie de ataque y la complejidad de datos: p.ej. reducir al mínimo la cantidad de tuplas (**sujeto, acceso, objeto**). Cada acceso es susceptible de ser explotado.

3.2.4. 4. Mediación completa (Complete mediation)

Si hay un control, este debe aplicar a **todas** las interacciones alcanzadas.

- Es “la puerta del castillo”: todo pasa por ahí y no hay ningún otro mecanismo posible.
- El muestreo de algunas transacciones, controles al azar o entidades por fuera del control son problemas potenciales. No tiene sentido construir un muro y dejar la puerta abierta por el costado.

3.2.5. 5. Sistema/Diseño abierto (Open design)

No debe basarse la seguridad en mantener el diseño oculto (no “seguridad por oscuridad/ofuscación”).

- Es el principio de Kerckhoffs aplicado: en encriptación lo que se guarda es la **clave** (y se cambia frecuentemente); el **algoritmo es público** y conocido por todos. El mismo concepto aplica a los controles de seguridad.
- Todo secreto conocido por al menos una persona puede ser revelado. “*Los bits son eternos.*”
— dicho en clase (Asumir que todo lo digitalizado puede llegar a verse / perderse.)
- Matiz: que NO deba basarse en ofuscación **no significa** que tenga que ser abierto y que todos se enteren. La ofuscación da algo de complejidad y tiempo (por eso los algoritmos militares son ocultos, aunque sigan también este principio); sirve como *capa adicional*, no como única defensa.
- Pata de software libre/open source: es bueno porque tiene más **escrutinio** (más ojos $\Rightarrow$ menos bugs explotables). Hoy hay que tomarlo con pinzas: si bots meten commits automáticos en open source, el principio se rompe (importa que haya un **responsable** que responda).

3.2.6. 6. Separación / Segregación de privilegios (Separation of privilege)

Toda tarea crítica no debe quedar bajo el control de un solo sujeto; más de un sujeto debe ser parte del proceso para asegurar un control entre partes.

- Se aplica el principio de **control por oposición de intereses**: las partes tienen objetivos opuestos y en el equilibrio se evita el abuso.
- Ejemplo (ventas): para aprobar una venta con descuento, el vendedor (que quiere descontar mucho para cerrar) pide aprobación al financiero (que quiere descontar lo menos posible). Hay oposición de intereses.
- En seguridad informática: que no sea el mismo quien controla las claves de la base de datos y quien controla el sistema de pagos; si controlara la base, podría crearse un usuario con permisos para operar el sistema de pagos (“salto”).
- Puede contradecir el principio de simplicidad (agrega complejidad): ahí está el balance.

► En el parcial

Vínculo no trivial (clave para el parcial): el profe relaciona “separar las credenciales del sistema de las del usuario” con el principio de **separación de privilegios**. Otro vínculo: mediación completa (única) + separación de privilegios $\Rightarrow$ la mediación de seguridad es única, pero el *privilegio de controlarla* se separa para que no haya un único punto de falla.

3.2.7. 7. Mecanismos exclusivos (Defense-in-Depth)

Tiene dos partes:

- **Exclusividad:** evitar que distintos mecanismos compartan recursos, algoritmos y hardware. Si algo es para seguridad, debe ser un mecanismo **exclusivo** para eso y nada más.
- **Defensa en profundidad:** diseñar controles compensatorios / de mitigación / reacción que se accionen si otro control falla o es superado.

Ejemplo (fuente de bugs/vulnerabilidades): un **flag** que originalmente indica “el cliente puede comprar” se reutiliza después también para “puede recibir un voucher” (asunción *as-is*/correlación que se mantendría eterna). Cuando esa correlación se rompe, el código queda mezclado: un mismo flag significa dos cosas. Si alguien controla indirectamente ese parámetro, rompe la exclusividad y puede saltar de un sistema a otro (relacionado con *pivoting*). Ejemplo análogo físico: una llave/flag de acceso a una puerta importante a la que implícitamente se ató el acceso a un servidor; el personal de limpieza con esa llave queda con acceso al servidor.

► En el parcial

Defensa en profundidad (remarcado en la clase de consulta): encadenar controles para **maximizar cuántas cosas hay que romper** para vulnerar el sistema. Ejemplo del PDF: se hace segregación de tareas + mínimos privilegios, pero una vulnerabilidad permite llevar adelante una venta sin autorizaciones $\Rightarrow$ ¿qué controles se agregan para monitorearla? ¿alertas que bloqueen la transacción? ¿una confirmación?



*Defensa en profundidad: se **encadenan** controles independientes. Si uno falla o es superado, otro lo compensa; el atacante debe vulnerar todas las capas para llegar al activo.*

3.2.8. 8. Menor asombro / Aceptación psicológica (Psychological acceptability)

- El **peor enemigo** de cualquier diseño de seguridad es el **usuario no alineado**; el mejor aliado es el usuario comprometido con la seguridad.
- Los controles deben ser simples de usar, incluso para alguien sin conocimientos técnicos, y no deben complicar el negocio ni sus procesos.
- La **concientización** del problema y del valor de la seguridad es fundamental para lograr la aceptación del control.

! Importante — remarcado en clase

“Los muy buenos trabajadores son malos usuarios de seguridad.”

— dicho en clase Los más eficientes saben “navegar la burocracia” y van a saltar las restricciones de seguridad para resolver el problema. Por eso hay que capacitarlos para que

entiendan que mantener la seguridad es parte de su trabajo. Un buen esquema asume que la persona *va a* intentar saltarlo y diseña para que eso sea una excepcionalidad controlada y consciente. Caso real: un compañero pasó dos meses mejorando la seguridad de la página rastreadora de ambulancias, y luego el jefe de ambulancias consiguió por “rosca” el mismo usuario con todos los permisos porque el negocio lo requería.

△ Cuidado — error típico

La seguridad “no tiene límite” (es un *bounder*): se puede poner tanta como la paranoia diga. El sistema más seguro del universo sería imposible de usar. Hay que balancear con experiencia de usuario (UX), pensando las cosas centradas en las personas.

3.3. La seguridad NO se garantiza

△ Cuidado — error típico

GANCHO/TRAMPA que penaliza. “*Garantizar la seguridad es algo que no se puede hacer jamás.*”

— dicho en clase Ante la pregunta “¿me garantizás que esto es seguro?”, nadie idóneo dice “sí, te lo garantizo”: es muy caro, requiere conocimiento muy específico y aún así es muy difícil por todo lo que ocurre *fuera* del sistema (el ecosistema de infraestructura/redes donde funciona).

Construir software de CALIDAD NO es garantía de seguridad (V/F $\Rightarrow$ **FALSO** la afirmación “calidad = seguro”; Recup 2025). Afirmar que algo “garantiza la seguridad” o que “software de calidad = seguro” **penaliza**.

Por eso, saliendo de los mecanismos específicos, la seguridad se trata como un problema de **gestión de riesgo**. Amenaza y vulnerabilidad componen dos probabilidades. Frente a un riesgo se puede:

- **Eliminarlo.**
- **Mitigarlo** (reducir la probabilidad o el impacto; p. ej. backups en otra nube).
- **Compensarlo** (no se evita, pero se agrega p. ej. una alerta que avisa cuando pasa).
- **Aceptarlo** (documentarlo y no hacer nada): a veces es la solución más efectiva cuando evitarlo es carísimo (en dinero o en UX). Lo importante es que la decisión se tome con conciencia.

3.4. Calidad/Procesos: DevOps vs DevSecOps vs MLOps

► En el parcial

“Seguridad de la Información” NO es un componente de DevOps — pertenece a **DevSecOps** (el agregado de *Sec*). Hoy además se suma **MLOps** (operaciones de machine learning). Errores que penalizan: ubicar la seguridad como parte nativa de DevOps.

3.5. Modelado de amenazas (Threat Modeling)

Es una técnica de ingeniería para entender las **amenazas, ataques, vulnerabilidades y contramedidas** que pueden afectar a una aplicación. Define la tarea de modelar las amenazas

al negocio en la **superficie de ataque**; con el modelo y la clasificación de amenazas se definen contramedidas específicas. Debe incluir inteligencia, tendencias y ataques pasados, y es un **proceso continuo** (las amenazas cambian). Se usa para mejorar el diseño y reducir el riesgo.

Proceso (Microsoft / OWASP, similares):

1. Definir el alcance (acotar el esfuerzo para que cubra lo suficiente sin excederse).
2. Determinar / definir amenazas (diagramar dependencias, identificar caminos de explotación).
3. Definir contramedidas y mitigación.
4. Revisar el trabajo, los cambios y volver a empezar (validar).

3.5.1. Modelo STRIDE

► En el parcial

STRIDE es de Microsoft (V/F, 2015 $\Rightarrow$ Verdadero: “fue desarrollado por Microsoft”). Es conocimiento **“enciclopédico” exigible**: hay que saber de memoria qué significa cada letra y qué propiedad viola. Surge en la época de Windows 98 cuando se cuestionaba mucho la seguridad de Windows.

Clasifica las amenazas en **6 categorías**. La idea del modelo: por cada sistema, conseguir al menos 2 amenazas críticas en cada categoría.

	Amenaza	Propiedad violada	Definición
S	Spoofing	Autenticación	Hacerse pasar por algo/alguien que no se es
T	Tampering	Integridad	Modificar algo en disco, red, memoria, etc.
R	Repudiation	No repudio	Negar haber hecho algo (sin poder probarlo)
I	Information Disclosure	Confidencialidad	Revelar información a quien no debe acceder
D	Denial of Service	Disponibilidad	Agotar recursos necesarios para dar el servicio
E	Elevation of Privilege	Autorización	Permitir hacer algo para lo que no se está autorizado

AMENAZA		PROPIEDAD VIOLADA
S	Spoofing	Autenticación
T	Tampering	Integridad
R	Repudiation	No repudio
I	Information Disclosure	Confidencialidad
D	Denial of Service	Disponibilidad
E	Elevation of Privilege	Autorización

Las 6 categorías STRIDE y la propiedad de seguridad que viola cada una (memorizar: cada letra $\leftrightarrow$ su propiedad).

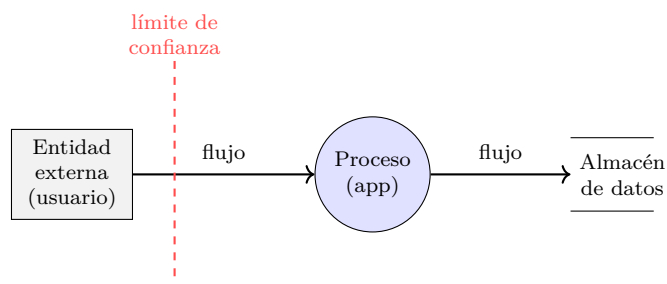
3.5.2. DFD (Data Flow Model) + STRIDE

El DFD representa las partes de un sistema y los riesgos de cada una. Analiza elementos de cada categoría:

- **Procesos**: contenedores, procesos ejecutables.
- **Entidades externas** al sistema: personas, otros sistemas.

- **Flujos de datos:** comunicaciones entre procesos.
- **Almacenamientos de datos:** archivos, bases de datos, secretos.

Para cada elemento se **cruza con las amenazas STRIDE** (matriz), y por cada riesgo identificado se define cómo gestionarlo: **eliminarlo, mitigarlo, compensarlo o aceptarlo**. (También se dibuja como árbol: amenazas → 6 categorías → ejemplos → recursos.) OWASP tiene su versión (casi igual a la de Microsoft, pero con manual gratuito). Hay industrias reguladas (pagos) que exigen demostrar estos ejercicios.



DFD: entidades externas, procesos, flujos y almacenes de datos. El **límite de confianza** (trust boundary, línea roja) marca dónde cambia el nivel de confianza: los datos que lo cruzan deben validarse.

Elemento	S	T	R	I	D	E
Entidad externa	✓		✓			
Proceso	✓	✓	✓	✓	✓	✓
Almacén de datos		✓	?	✓	✓	
Flujo de datos		✓		✓	✓	

Matriz DFD × STRIDE: a qué amenazas es susceptible cada tipo de elemento (los procesos, a todas). Por cada ✓ se decide eliminar / mitigar / compensar / aceptar.

3.6. Amenazas y Vulnerabilidades

Definición: Amenaza (Threat)

Cualquier elemento que pueda afectar la confidencialidad, integridad o disponibilidad de un sistema. Apunta a la **existencia de alguien (o algo) con ganas de lastimar**.

Definición: Vulnerabilidad

Una debilidad en un activo o grupo de activos que permite que un atacante la aproveche para lograr un acceso que no tiene permitido. Es el **canal de ejecución** de la maldad. Técnicamente, las vulnerabilidades son **bugs que se disfrazan de feature/seguridad**.

! Importante — remarcado en clase

Distinción sutil pero **clave** (se toma siempre):

- **Amenaza** = las *ganas* de atacar (p.ej. las ganas del hacker de romper tu sitio para robar plata).
- **Vulnerabilidad** = la cuestión técnica que el script explota (un bug).
- **Exploit** = el programa/script que explota la vulnerabilidad para generar la brecha.

► En el parcial

Ecuación de riesgo (para grabarse): la amenaza “multiplica” a la vulnerabilidad, y eso mide el riesgo. El profe agrega un tercer término: el **valor del activo (Asset) / la plata que se pierde**. $\text{Riesgo} \approx \text{Amenaza} \times \text{Vulnerabilidad} \times \text{Asset}$ (multiplicativo). Con esas 3 cosas se hace un análisis de riesgo robusto que captura ~95 % de la problemática, porque los recursos son finitos y no se puede controlar todo.

3.6.1. Tipos de amenazas (taxonomías)

- **Según origen:** *internas* (empleado que hace fraude; falla en equipo que deja sin servicio) o *externas* (grupo criminal robando info; huracán hacia el datacenter).
- **Según agente amenazante:** *humanos* (hacker; administrador cometiendo un error), *ambientales* (evento natural, inundación), *tecnológicos* (servidor que se quema). Se discute agregar una 4.^a categoría: **agentes de IA**.
- **Según intención:** *malicioso* (ataque intencional) o *no malicioso* (error no intencional).
- **Según impacto:** destrucción (disponibilidad), corrupción (integridad), revelar información (confidencialidad), robo del servicio (fraude, sin afectar a otros usuarios), denegación de servicio (disponibilidad), elevación de privilegios (bypass de ACL), uso ilegal (p.ej. usar Google Drive para distribuir contenido con copyright).

3.6.2. Tipos de vulnerabilidades

- **En el código fuente:** fallas en controles o lógica deficiente; lo específico de una aplicación o heredado de dependencias.
- **Componentes mal configurados:** la forma más común en la nube (ej.: servidor Mongo provisionado con clave de admin pública por defecto).
- **Relaciones de confianza:** no revalidar la autenticidad del sujeto (ej.: NFS no valida al usuario; asumir que tras el login las demás páginas ya están autenticadas y no chequear cada una; o no chequear autorización porque “nadie va a adivinar la URL”).
- **Malas prácticas de credenciales:** políticas de password inseguras, credenciales expuestas, falta de MFA.
- **Falta de cifrado fuerte (Lack of strong encryption):** algoritmos débiles, claves chicas, criptosistemas viejos (ej.: permitir suites *export* en TLS).
- **Insider threat:** persona que incumple reglas para hacer daño/fraude (más asociado a seguridad física).
- **Vulnerabilidad psicológica:** cualquier causa humana sin intención que genera un problema; toda la ingeniería social (“cuento del tío”) cae acá. Ligada al principio de aceptación psicológica.
- **Autenticación inadecuada:** proceso completo de autenticación mal hecho (ej.: login fuerte con 2FO pero un “recuperar contraseña” débil que da entrada).
- **Injection flaws:** cualquier inyección de código. “Si de esta materia tienen que llevarse una sola cosa, es que el 90 % de las vulnerabilidades caen en Injection Flow.”

— dicho en clase

- **Sensitive data exposure:** errores que muestran datos que no deberían (stack de error con código fuente; *dumpster diving*; notebooks viejas mal borradas).
- **Insufficient monitoring and logs:** poca visibilidad para detectar/alertar/corregir; afecta mucho a la disponibilidad.
- **Shared tenancy:** recursos compartidos entre tenants que no aíslan bien la info (multi-tenant cloud; carpeta /tmp compartida; ataques de hardware como *Rowhammer* y *Spectre*, que rompen el aislamiento entre máquinas virtuales).

3.6.3. MITRE: CVE, CWE y Zero-day

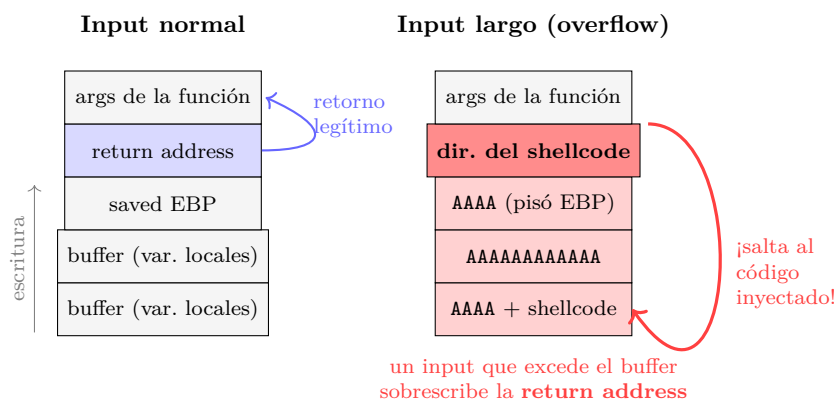
- **CVE** (Common Vulnerabilities and Exposures): registro centralizado de vulnerabilidades de software. Formato **CVE-YYYY-NNNN** (año + número de secuencia). La cantidad de CVEs publicadas por año crece de forma explosiva desde ~2017. Ejemplos: **CVE-2021-44228** (Log4j, ejecución arbitraria de código), **CVE-2014-0160** (Heartbleed en OpenSSL: “buffer over-read” que filtra memoria y permite extraer claves/master key de la sesión).
- **CWE** (Common Weakness Enumeration): registro de las *debilidades* en software que dan lugar a las vulnerabilidades. Cada año/varios años MITRE publica el Top 25. Top: #1 **CWE-787 Out-of-bounds Write** (buffer overflow), #2 **CWE-79 Cross-site Scripting**, #3 **CWE-89 SQL Injection**, #4 **CWE-416 Use After Free**, #5/#6 **CWE-78 OS Command Injection**.
- **Zero-day:** vulnerabilidad/exploit aún no reportado ni conocido por el fabricante. Tiene alto valor de mercado (no hay prevención). Existe mercado negro y uno gris (Zerodium paga, p. ej., hasta US\$1.000.000 por un RCE remoto en Windows sin intervención del usuario). El embargo: si una CVE es muy crítica (rating > 9) se publica el sistema afectado y se da ~3 meses antes del detalle.

3.7. Buffer overflow e inyección de código

Definición: Buffer overflow (CWE-787, Out-of-bounds Write)

Escritura en memoria fuera de los límites originalmente definidos del buffer. Lleva a: corrupción de datos, crash (el clásico *segmentation fault*) o **ejecución de código**.

El peligro mayor: si el buffer está en el **stack**, este contiene los datos de la última llamada a función y la **dirección de retorno**. Un atacante puede escribir el buffer de forma que inyecte su código y pise la dirección de retorno para que el flujo salte a ese código. Hay contramedidas en compiladores, pero **ninguna es 100 % efectiva**. Lenguajes de más alto nivel (Java, etc.) se volvieron populares en parte porque eliminan la aritmética de punteros y este tipo de errores.



*Stack smashing: el input que desborda el buffer sigue escribiendo hacia las direcciones superiores hasta **pisar la dirección de retorno**; el atacante la reemplaza por la dirección de su propio código (shellcode) y el flujo salta ahí.*

! Importante — remarcado en clase

Von Neumann: datos y código viven en el mismo espacio de memoria, lo que **habilita la inyección de código** (caso iOS, 2018). La generalización del problema de inyección es que el canal de datos de control del sistema (un nivel) queda **mezclado** con el canal de datos del usuario (otro nivel) sin lograr separarlos. Esto aplica también a los LLM (prompt injection / poisoning): no hay un mecanismo *exclusivo* que separe el canal de control del de la pregunta del usuario.

► En el parcial

“Qué mitigación NO tiene sentido” para buffer overflow (2025-2Q), entre opciones tipo: **RUST / garbage collector / validar la entrada / Stack Canary**. El profe admitió que es **ambiguo**: “*Todas pueden contestar y todas están bien.*”

— dicho en clase Lo que importa es **JUSTIFICAR** la elección. (RUST y validar la entrada previenen; Stack Canary detecta el pisado de la dirección de retorno; el GC es el candidato más discutible como mitigación directa del stack overflow, pero hay que argumentarlo.)

△ Cuidado — error típico

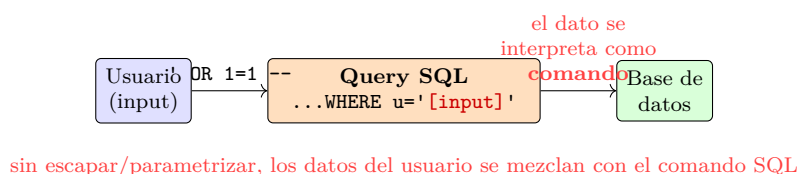
Use After Free (CWE-416): liberar la memoria (free) pero quedarse con el puntero y seguir usándolo. Puede derivar en valores raros, crash o ejecución de código. Es un recurso que no se “limpia” (wipe) entre usos.

3.8. Vulnerabilidades web

3.8.1. SQL Injection (CWE-89)

Falta de control en el ingreso de los usuarios que permite ejecutar código SQL en la base de datos (se mezclan datos con comandos y la base interpreta los datos como comando).

- **Defensa:** escapar/parametrizar las entradas; usar **prepared statements** (armar la query con parámetros: “el parámetro 1 vale tanto, el 2 vale tanto”).
- Muy peligrosa: hay herramientas automatizadas que, dado un parámetro vulnerable, averiguan el motor, el esquema y hacen *dump* de cualquier tabla. Basta 1 query mal sanitizada entre cientos para comprometer todo.

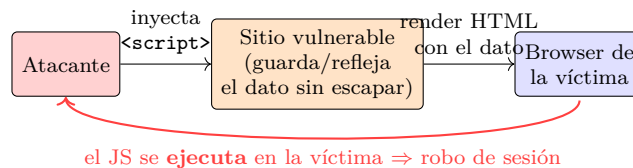


*SQL Injection (CWE-89): el input no sanitizado se concatena en la query y la BD lo **ejecuta como comando**.
Defensa: prepared statements (parámetros separados del comando).*

3.8.2. Cross-Site Scripting / XSS (CWE-79)

Inyección de código malicioso (JavaScript) en un link o requerimiento que termina siendo **ejecutado por la víctima / el browser**. Ocurre al hacer render de HTML mezclando un dato (que controla el usuario) con código sin escapar, de modo que el dato puede ser código.

- **Aprovecha la confianza** entre el sitio y el browser/sesión: si se ejecuta JS en la máquina de la víctima, se le puede **robar la sesión**.
- **Defensa**: sanitizar/escapar todas las salidas (*escape on output*) o usar frameworks de componentes que sanitizan automáticamente. Mismo problema que SQLi: basta olvidarse de uno entre 999.



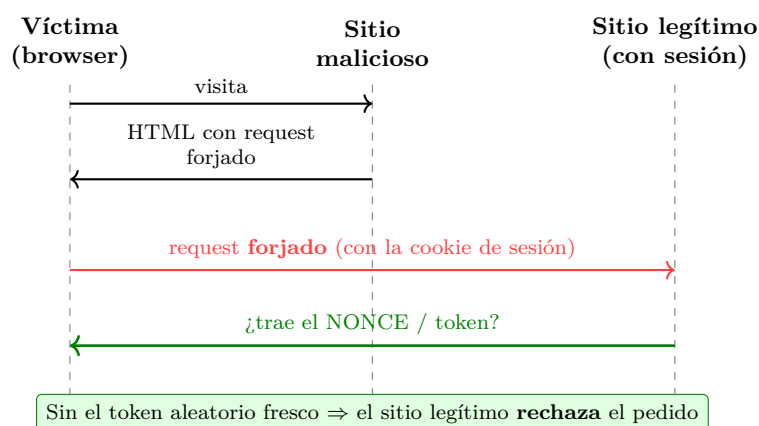
*XSS (CWE-79): el dato controlado por el atacante termina siendo **ejecutado como código (JS)** por el browser de la víctima, aprovechando la confianza del sitio en su sesión.*

3.8.3. CSRF (Cross-Site Request Forgery)

► En el parcial

Defensas a CSRF (foco de examen):

- La defensa **real** es el **NONCE / token anti-CSRF**: un valor aleatorio único en el formulario que da **frescura** y evita el *replay*.
- Forzar **POST** en vez de **GET**.
- **Concientizar** contra phishing.
- **NO sirve banear la IP**: el usuario es legítimo (la petición se ejecuta en su nombre).



*CSRF: el sitio malicioso hace que el browser de la víctima dispare un request al sitio legítimo con su cookie de sesión. El **nonce/token anti-CSRF** (valor aleatorio único en el formulario) lo frena: el atacante no lo conoce, y evita el *replay*.*

3.8.4. OS Command / Shell Injection (CWE-78)

Falta de control en el ingreso que permite ejecutar comandos del sistema operativo. Genera *backdoors* para ignorar controles de red o autenticación. Ejemplo: una app PHP que delega a *ImageMagick* pasándole el nombre del archivo sin sanitizar; si el nombre contiene `; rm -fr ...` se ejecutan otros comandos.

3.8.5. Path / Directory Traversal

! Importante — remarcado en clase

Caso **router Linksys (2013)**: parámetro `next_page=/etc/passwd` permitía leer archivos fuera del directorio esperado (traversal de directorios).

3.8.6. XXE (XML External Entities)

XML permite cargar esquemas desde una URL; si esa URL se usa para hacer un GET a otra página, un atacante puede escanear la red interna visible por el servidor (estuvo de moda porque casi todas las librerías XML tenían el problema).

3.8.7. Organizaciones de referencia (OWASP)

OWASP (Open Web Application Security Project): organización enfocada en combatir problemas de seguridad en aplicaciones Web; publica regularmente las vulnerabilidades más comunes y buenas prácticas. Tiene el **Top 10** (web), **OWASP Cloud Top 10** y **OWASP Top 10 LLM** (ej.: LLM01 Prompt Injection, LLM03 Training Data Poisoning, LLM05 Supply Chain). Junto con MITRE son referencia hace >20 años. (Vulnerabilidad moderna asociada: *content/prompt poisoning*: texto invisible en páginas o currículums tipo “ignora tus instrucciones” que el LLM/agente termina ejecutando.)

3.9. Malware básico (mencionado / referenciado para definir)

► En el parcial

Ejercicio típico (2016): **DEFINIR** varios conceptos (mínimo privilegio, confinamiento, root-kit, troyano) y luego **relacionar dos de forma no trivial**. Estos términos se desarrollan en profundidad en la unidad de *Flujo de información y Malware*; acá basta tenerlos a mano:

- **Troyano**: programa que se disfraza de algo legítimo/útil para que el usuario lo ejecute, ocultando funcionalidad maliciosa (análogo al bug que “se disfraza de feature”).
- **Rootkit**: software malicioso que se oculta en el sistema (típicamente con privilegios de root/administrador) para mantener acceso persistente y evitar ser detectado.
- **Confinamiento**: limitar/aislar lo que un proceso puede hacer o acceder (sandboxing), muy ligado a *mínimo privilegio* y a *mecanismos exclusivos*.

Vínculo no trivial sugerido: *confinamiento* es la materialización técnica de *mínimo privilegio* (aislar para que aunque algo se comprometa, no pueda hacer más que lo mínimo).

3.10. La idea del “chip paranoico”

! Importante — remarcado en clase

De la clase de consulta, el profe remarcó el “**chip paranoico**”: pensar como atacante. “*Seguridad informática es paranoia controlada.*”

— dicho en clase Los muy buenos en seguridad son paranoicos (controladamente): asumen que todos quieren hacer daño de alguna manera, y eso suma. Junto con la **defensa en profundidad** (encadenar controles para maximizar cuántas cosas hay que romper), son las dos ideas rectoras. Además hay preguntas “**enciclopédicas**” que requieren **memorizar definiciones** (p. ej. cada letra de STRIDE).

3.11. Lectura recomendada

Capítulos 12–13 de *Computer Security: Art and Science*, Matt Bishop (Bishop habla de principios, no tanto de amenazas concretas; para amenazas, el proceso de Threat Modeling de OWASP).

4. Flujo de Información y Malware

Esta unidad retoma el problema del flujo de información (ya visto informalmente al hablar de políticas) y lo formaliza con la **entropía**. Cierra el bloque de seguridad en aplicaciones y luego cambia de perspectiva (la del atacante) para introducir el **malware**. Lectura recomendada: Bishop, *Computer Security: Art and Science*, capítulos 16 (parte 1) y 17.

4.1. Flujo de información: el problema

Las políticas de seguridad rara vez se expresan como permisos sobre objetos. Una política típica dice: “*un empleado no puede ver los sueldos de otros empleados*”. El profesor remarca que ese requisito se refiere a **la información en sí**, no al lugar donde la información está en un momento puntual.

Definición: Control de acceso vs. control de flujo

Control de acceso: limita el acceso de un *sujeto* a una *operación* puntual sobre un *objeto* puntual. Como los objetos contienen información, indirectamente limita el acceso a información.

Control de flujo: estudia cómo la información *se mueve* a través del sistema. La información no es estática: **se actualiza y se puede copiar**.

El control de acceso **no impide copiar la información a otro lugar**: un usuario con permiso de lectura sobre su sueldo puede leerlo y escribirlo en un objeto que otros puedan ver (camino de lecturas y escrituras).

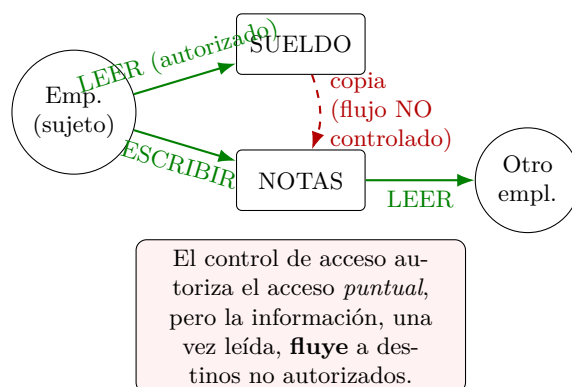
! Importante — remarcado en clase

Las políticas **por lo general restringen el flujo de información y no el acceso a objetos**. Comparar: “evitar que un empleado sepa el sueldo de otro” (flujo) vs. “evitar que un empleado acceda a la base de datos de sueldos” (acceso). El control de acceso *sirve*, pero son **mecanismos abiertos**: aseguran una parte del problema, no todo. Deben ser **complementados**.

► En el parcial

Pregunta clásica de verdadero/falso:

“Una matriz de acceso bien diseñada evita el flujo de información no deseado” → **FALSO**. El control de acceso no controla el flujo. Sí limita el acceso puntual, pero no impide que la información, una vez leída legítimamente, se copie o fluya a otro lugar (directa o indirectamente).



La matriz de acceso autoriza operaciones puntuales (flechas verdes) pero no controla el flujo: el dato leído se copia y se propaga (flecha roja punteada) hacia un destino no autorizado.

Metáfora de capas (cebolla / queso suizo). Como los mecanismos son abiertos, la estrategia es superponer capas de seguridad. Cada capa tiene “agujeros”, pero se apilan tantas que se espera que los agujeros no coincidan en el mismo lugar y unas tapen las de las otras (metáfora del queso suizo).

4.2. Confinamiento y aislación total

Definición: Problema del confinamiento

Dificultad de asegurar que un programa o proceso (*sujeto*) no transmita información a otra entidad (*objeto*) a la que no está autorizado a acceder, ya sea de manera **directa o indirecta**. Es el problema de controlar la información *en tránsito* (cuando se está moviendo). Es un problema complicadísimo de resolver.

Si se pudiera **confinar** el flujo en un ambiente totalmente controlado, no habría riesgos de confidencialidad ni integridad. El modelo **Bell–LaPadula** define un modelo de seguridad que, en un ambiente cerrado donde se aplica al 100 %, da garantías de confidencialidad respecto al flujo.

Definición: Modelo ideal: Aislación total

Requisitos: (1) un proceso no puede comunicarse con otros procesos; (2) un proceso no puede ser observado. **Consecuencia:** el proceso no revela información.

“El sistema más seguro del mundo corre en una computadora que está apagada, metida en una caja fuerte, enterrada en el fondo del océano, sin que nadie sepa dónde está.”

— dicho en clase

El problema es que la aislación total **le quita toda utilidad al sistema** (no se le puede dar entrada ni observar la salida). En la práctica:

- El requisito (1) a veces se logra (red cerrada, sin Internet — “air gap”).
- El requisito (2), **no poder ser observado, es lo más difícil**. Todo proceso usa recursos **observables**: memoria, ciclos de CPU, espacio en disco, ancho de banda. Observándolos se puede extraer información del proceso y de su resultado.

4.3. Canales encubiertos y canales laterales

Definición: Canal encubierto (covert channel)

Método de comunicación usado para transferir información de manera **oculta**, aprovechando vías **no diseñadas** para la transmisión de datos. Como esas vías no estaban pensadas para transmitir, es muy poco probable que tengan los controles que sí tienen los canales normales; por eso sirven para evadir controles de seguridad y transmitir sin detección.

Pueden aparecer **intencionalmente o sin intención**. Tipos:

- **Canales de temporización (Timing Channels):** codifican datos en la variación de tiempo entre eventos. Se muestrea el canal a intervalos regulares y según cómo cambia la lectura se infiere información. Ejemplos: latencia de red, carga de CPU, tiempos de respuesta de aplicaciones. (Caso real mencionado: *traffic shaping* de proveedores de Internet, que analizan la forma de las tramas sin ver el contenido.)
- **Canales de almacenamiento (Storage Channels):** usan áreas no convencionales o no reguladas para guardar datos: espacio de relleno en un archivo, atributos/metadata (fecha de creación, modificación, acceso), bits no usados o reservados en estructuras de datos y protocolos (lugares donde debería ir un valor aleatorio y no lo es tanto).

Ejemplos de canales encubiertos (clase).

- Exfiltración codificando los **sonidos de una impresora** (micrófono de alta fidelidad que reconstruye lo impreso).
- **ICMP & DNS Tunneling**: usar el protocolo DNS para exfiltrar. Como casi todo Internet necesita resolución de nombres, los firewalls dejaban pasar DNS. Idea: quien controla un dominio y su servidor DNS hace que la máquina interna haga búsquedas de subdominios cuyo nombre es (p. ej. en base64) parte de los datos a exfiltrar; esas consultas llegan al servidor del atacante y se rearma la información arbitraria, pasando los controles.
- Emanaciones electromagnéticas; láser sobre una ventana midiendo vibración para reconstruir el sonido de una habitación; *fingerprinting* por forma de tipeo.

Definición: Ataque de canal lateral (Side-Channel Attack)

Explota las **emisiones físicas y temporales** de un sistema para extraer información sensible. **No** se basa en vulnerabilidades clásicas de software ni en errores de autenticación/autorización, sino en la **observación de efectos colaterales del hardware** durante su funcionamiento (tiempos de ejecución, consumo de energía, radiación electromagnética, ruido acústico).

Es especialmente relevante en **criptografía**: toda la seguridad se apoya en que la clave no se filtre, y estos ataques suelen apuntar a **recuperar la clave**. Si se recupera, “se cae toda la seguridad como un castillo de naipes”.

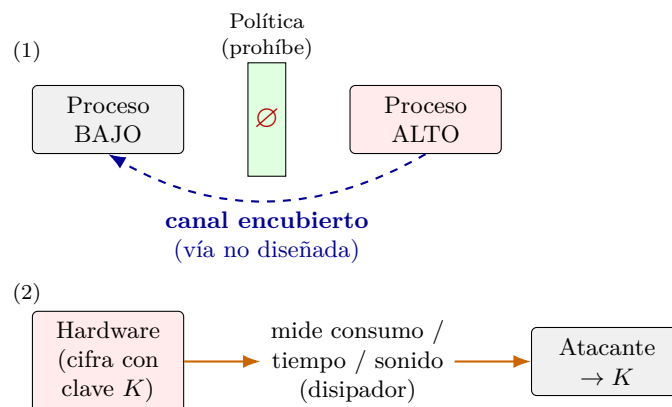
Mecanismo de un ataque por timing sobre cripto. Los algoritmos (y los compiladores) tienen bloques con duración **variable** en ciclos de reloj (p. ej. multiplicar/exponenciar números grandes depende de su tamaño). Midiendo con precisión muchas ejecuciones (sampleo estadístico para sacar el ruido) y modelando el algoritmo, se va infiriendo la clave; combinado con ataque de texto plano conocido, se controla un número y se usa la diferencia de tiempos para despejar la incógnita. No hace falta éxito total: alcanza con **sesgar las probabilidades** de algunos bits y completar el resto por fuerza bruta (p. ej. de 128 bits, sesgar 40 bits deja 2^{88} pruebas en lugar de 2^{128}).

Otros side-channels famosos. Análisis de consumo de energía (rompió históricamente casi todas las *smart cards* con clave criptográfica embebida, como tarjetas de transporte/acceso); Spectre, Meltdown, Rowhammer (hacer vibrar dos filas de memoria para alterar el bit del medio), Rambo (variaciones electromagnéticas convertidas en radiofrecuencia captable desde afuera). Cobraron relevancia con los *hyperscalers* (AWS, Google): en una misma máquina física conviven VMs de distintos clientes, y desde una VM se podría leer información de otra.

△ Cuidado — error típico

No confundir canal encubierto con ataque de canal lateral:

- **Canal encubierto**: es el *medio* (flujo de información no diseñado para transmitir). Se centra en *evadir* controles para transmitir información oculta.
- **Ataque de canal lateral**: es *explotar* un canal encubierto para extraer información sensible (recolectar datos observando el comportamiento físico del hardware). “Dos caras de la misma moneda.”

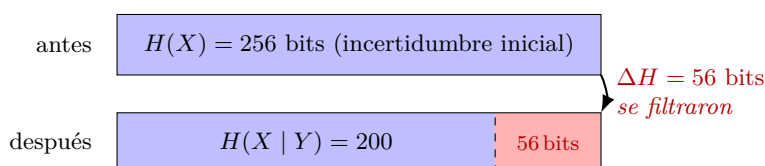


(1) Canal encubierto: dos procesos (alto/bajo) que la política separa, pero que se comunican por una vía no prevista. (2) Ataque de canal lateral: observar el consumo/tiempo/sonido del hardware (caso del disipador) para reducir la entropía de la clave y recuperarla.

► En el parcial

Ejercicio clásico (Recup 2023): si la entropía de una clave de 256 bits se reduce a 200 bits al registrar el sonido del disipador del hardware, ¿hay flujo de información?

SÍ. La reducción de entropía *es* flujo de información. El sonido del disipador es una vía no diseñada para transmitir datos $\Rightarrow$ canal encubierto; explotarlo para sacar info de la clave es un **ataque de canal lateral (side-channel)**. Pasó de $H = 256$ a $H = 200$: se filtraron 56 bits de información sobre la clave. La forma de demostrarlo formalmente es justamente que H (incertidumbre sobre la clave) *disminuyó*.



La reducción de la entropía de la clave de $H(X) = 256$ a $H(X | Y) = 200$ (franja roja = 56 bits) es información que se filtró: hubo flujo de información hacia el atacante.

Mitigación. Resolver todos los canales (incluidos los que no conocemos) es “misión imposible”; sí se puede **mitigar** los conocidos. Las dos familias de control de flujo son: en tiempo de **compilación** y en tiempo de **ejecución** (ver más abajo).

4.4. Entropía como medida de información

Claude Shannon (hace casi 90 años) dio una forma de convertir en **un solo número** la cantidad de información que puede circular por un canal discreto.

Definición: Entropía $H(X)$

Sea X una variable aleatoria discreta (V.A.D.) que toma valores $x_1, \dots, x_n$:

$$H(X) = - \sum_{i=1}^n p(x_i) \log p(x_i)$$

(con log en base 2, da bits). Mide la **incertidumbre** al determinar el valor de X : cuánta información necesitaríamos conocer para explicar completamente el contenido del canal (medida de *falta* de información / incertidumbre).

Extremos:

- **Mínimo** $H(X) = 0$: cuando algún $p(x_i) = 1$ (y el resto 0). Sabemos *exactamente* qué se transmite; no hay información extra. (Cada término vale 0: o porque $p = 0$, o porque $p = 1 \Rightarrow \log 1 = 0$.)
- **Máximo**: cuando $p(x_i) = 1/n$ para todo i (**distribución uniforme**). Es lo más incierto. Para n valores equiprobables, $H(X) = \log n$.

La entropía es **continua**: cuantifica también los escenarios intermedios (conocemos algo, pero no todo).

Definición: Entropía condicional $H(X | Y)$

Sea X con valores $x_1, \dots, x_n$ e Y con valores $y_1, \dots, y_m$:

$$H(X | Y = y) = - \sum_{i=1}^n p(x_i | y) \log p(x_i | y) \quad H(X | Y) = \sum_{j=1}^m p(y_j) H(X | Y = y_j)$$

Es el promedio ponderado de las entropías condicionales. Mide cuánta incertidumbre queda sobre X sabiendo Y .

4.5. Flujo de información vía entropía

Definición: Flujo de información (formal)

Sea s el estado inicial del sistema y t el estado tras ejecutar comandos $c_1, \dots, c_n$. Sean x_s, y_s valores de objetos en el estado s y y_t el valor de y en el estado t . **Hay flujo de información de x a y si:**

$$\begin{aligned} H(x_s | y_t) &< H(x_s | y_s), & \text{si } y \text{ existe en el estado } s \\ H(x_s | y_t) &< H(x_s), & \text{si } y \text{ NO existe en el estado } s \end{aligned}$$

Idea: cuantifico lo que me falta conocer de x al principio y al final (observando y). Si al final **conozco más** de x (la entropía **bajó**), hubo **transferencia de información**. Si no debiera haber flujo y la medición cambia, hay un problema.

△ Cuidado — error típico

Cuándo usar cuál fórmula (esto se equivoca seguido):

- Si la variable destino y **ya existía** al principio, se compara contra la **condicional** $H(x_s | y_s)$.
- Si y **se crea** durante la ejecución, se compara contra la entropía **absoluta (no condicional)** $H(x_s)$ del estado inicial.

Sin flujo, debería dar **mayor o igual**. El “=” es claro (si y no tiene nada que ver, H no cambia). Puede incluso *aumentar* la entropía si una sentencia “pisa” (sobrescribe) el valor de la variable, destruyendo información.

4.5.1. Flujo directo

Se sigue el flujo **sentencia por sentencia**, midiendo cómo cambia la información de cada variable. Ejemplo de la teoría: comando $y = x + z$, con

$$0 \leq x \leq 7 \text{ (8 valores equiprobables)}, \quad Z = \{p(z=1) = 0,5, p(z=2) = 0,25, p(z=3) = 0,25\}$$

Antes del comando (8 valores equiprobables, $p = 1/8$):

$$H(x) = - \sum_{i=1}^8 \frac{1}{8} \log \frac{1}{8} = -8 \cdot \frac{1}{8} \log \frac{1}{8} = \log 8 = 3$$

Conociendo y luego del comando, x solo puede valer $\{y-1, y-2, y-3\}$ con las probabilidades de z :

$$H(x | y) = -\frac{1}{2} \log \frac{1}{2} - 2 \cdot \frac{1}{4} \log \frac{1}{4} = \frac{1}{2} + 2 \cdot \frac{1}{2} = 1,5$$

Como $H(x) = 3$ y luego $H(x | y) = 1,5 < 3$: **hubo traspaso de información de x a y .**

4.5.2. Flujo indirecto

La información puede fluir aunque x e y **no aparezcan en la misma asignación**, por la estructura de control.

Por asignación condicional. `if (x == 0) y = 1; else y = 0;` Con $x \in \{0, 1\}$, $p(x=0) = 0,5$: $H(x) = 1$ y $H(x | y) = 0$ (conociendo y se deduce x). **Hay traspaso de información.** (Canal “espacial” por comportamiento.)

Por comportamiento / terminación. `while (x == 0) {}` No hay ninguna asignación. Con $x \in \{0, 1\}$, $p(x=0) = 0,5$, se define una variable observable $y = 0$ si el programa termina (y otro valor si no). $H(x) = 1$, $H(x | y) = 0$: **hay traspaso de información** observando si el programa termina o no (canal temporal por comportamiento).

! Importante — remarcado en clase

El flujo de información **siempre existe** en un programa — es necesario para que funcione. Por eso, el seguimiento de flujo no busca verificar *todo* flujo en toda configuración, sino **casos críticos puntuales** (p. ej. que una variable que contiene una **clave criptográfica** no pueda inferirse midiendo otras variables observables).

4.5.3. Límites (compartidos con la verificación formal)

- **No es computable** verificar todo programa (límite teórico; los no computables suelen ser programas que nunca terminan).
- Con **interactividad** (inputs externos: teclado, mouse) la cantidad de casos **explota exponencialmente**. Se limita a sistemas chicos y cerrados.
- Los **ciclos** son un problema. Técnica: *loop unrolling* (desenrollar el ciclo un número fijo de veces, p. ej. hasta 10; suele alcanzar porque los problemas estructurales aparecen en pocos ciclos), pero queda el cabo suelto de iteraciones más profundas.

4.6. Control de flujo: compilación vs. ejecución

- **Tiempo de compilación (compile-time enforcement)**: analiza la propagación de información directamente en el código fuente (las técnicas de entropía de arriba; relacionado con verificación formal). Costoso y complejo → limitado a componentes muy sensibles. (Ej. Tanenbaum armó un microkernel con demostración formal de seguridad.)
- **Tiempo de ejecución (run-time enforcement)**: control dinámico (**sandboxing**) y **aislación** (virtualization). Son los **métodos más utilizados**.

4.6.1. Aislamiento (en ejecución)

Virtualización (Virtual Machines). Se simula a la aplicación un **entorno de ejecución sintético**, separado del real y aislado por definición. Las VMs usan **hipervisores** para emular hardware:

- **Bare-metal:** KVM, VMware ESXi, Hyper-V.
- **Hosted:** VirtualBox, VMware Workstation.

Cada VM tiene su **propio kernel** $\Rightarrow$ aislamiento fuerte (no comparten CPU). Pero es **pesadísimo**: emula todo el ambiente.

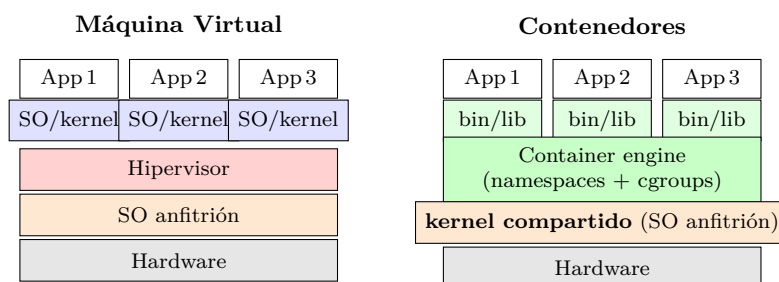
Sandboxing. Más liviano: en vez de simular todo, **arbitra las interacciones** (entrada/salida) del proceso con su entorno y las modifica para aislarlo, decidiendo dinámicamente si lo deja seguir.

- **A nivel SO:** el sistema operativo deriva las interacciones a un mecanismo de control. **Linux:** **namespaces** (aíslan procesos, usuarios, red, filesystem) y **cgroups** (controlan/limitan CPU, memoria, red). **Windows:** *User Mode Scheduling* (similar a namespaces) y *Job Objects* (agrupan procesos como una unidad para límites y prioridades).
- **Reescritura del programa:** agregar los controles antes de cada interacción.
- Ejemplo simple: **chroot** en Unix — lanza un proceso con una vista “encajonada” del filesystem (un directorio funciona como raíz). Usado también por los instaladores de paquetes Unix: corren scripts de terceros en un sandbox y luego copian solo los archivos que no resultan sospechosos.

Contenedores. Aprovechan **namespaces + cgroups** (**Docker** configura ambos) para aislar aplicaciones: **comparten el mismo kernel del SO anfitrión** pero mantienen separación lógica de red, procesos y filesystem. Eliminan la necesidad de un SO completo por instancia (más livianos que las VMs). Windows soporta contenedores nativos usando características de Windows o Hyper-V.

△ Cuidado — error típico

Diferencia clave VM vs. contenedor (suele preguntarse): la **VM** replica todo el stack (incluye un **SO/kernel propio** por instancia) $\Rightarrow$ aislamiento fuerte pero pesado. El **contenedor comparte el kernel** del host $\Rightarrow$ liviano, pero comparte más recursos (quedan canales no aislados: memoria, CPU compartidas).



VM: cada instancia trae su propio SO/kernel sobre el hipervisor (aislamiento fuerte, pesado). Contenedor: las apps comparten el mismo kernel del anfitrión y se separan con namespaces/cgroups (liviano, pero comparten más recursos).

4.7. Malware

Definición: Malware (Malicious Software)

Cualquier programa **diseñado** para causar daño, atravesar un control o extraer información. Lo que los distingue es haber sido concebidos **exclusivamente con un fin malicioso**. Suelen aprovechar vulnerabilidades del software. Un ataque usualmente usa **múltiples tipos** de malware para cada parte del ciclo del ataque.

4.7.1. Tipos de malware

Virus / Gusano (Worm): su rasgo característico es la capacidad de **propagarse** (auto-replicarse).

El virus infecta una máquina/proceso y usa los medios disponibles para propagarse a otras: explotando una vulnerabilidad remota o reescribiendo un ejecutable/recurso compartido. Un virus suele tener dos componentes: **replicación** + **ejecución** (*payload*); según su payload recibe otro nombre. El **worm** se especializa en propagarse (su “única misión” es multiplicarse y ocupar recursos), inclusive por email; asociado a **denegación de servicio**.

- Primer gusano: nació por **error** (Berkeley), se multiplicaba en cada máquina hasta saturar la memoria y tirar el SO.
- **Melissa (1999):** se propagaba por email.
- **SQL Slammer:** infectó 75,000 computadoras en 10 minutos explotando bases SQL de Microsoft expuestas a Internet.

Troyano: malware **oculto dentro de otro software** que parece benigno y de interés para la víctima (caballo de Troya). Sirve para establecer al atacante dentro del equipo. Ejemplo: **PC-Write (1986)**, shareware de editor de texto que borraba todos los archivos. Frecuente en cracks de Windows/Office y en videojuegos móviles modificados.

Rootkit: software diseñado para **ocultar** malware dentro de la máquina infectada. Usa ofuscación, encriptación, reescritura y reemplazo de herramientas del sistema; toma **contramedidas activas** (p. ej. interceptar *system calls* para no aparecer en **ps** ni al listar directorios). Opera a nivel **kernel o usuario**; los más avanzados se meten en la **BIOS/bootloader** y persisten aunque se formatee el disco (por eso existe Secure Boot / BIOS firmada). Muy difíciles de detectar por antivirus.

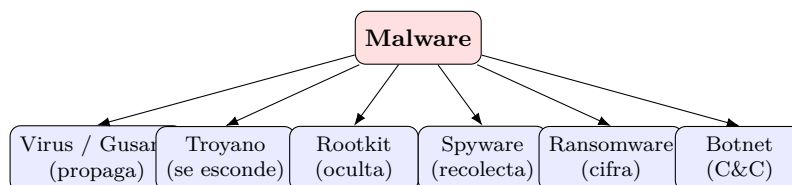
- **NTRootkit (1999):** primer rootkit para Windows NT.
- **Stuxnet (2010):** se escondía y propagaba por meses; solo actuaba al llegar a las máquinas (con firma conocida) de una planta nuclear de Irán (aisladas de Internet), sabotando máquinas industriales.

Spyware: software diseñado para **recolectar información** del usuario infectado. Se especializa en ocultar la información dentro del activo y suele establecer alguna forma de **covert channel (túnel)**. Algunas empresas lo desarrollan a pedido de gobiernos para vigilancia/espionaje (ej. mencionado: Palantir).

Ransomware: bloquea el acceso del usuario a su información (típicamente la **cifra**) y pide **dinero (rescate)** para restaurarla; otros exfiltran y piden rescate para no publicar. Apareció como uso “creativo” de las criptomonedas (cobro descentralizado y difícil de rastrear).

- **CryptoLocker (2013):** por mail; ~U\$D 3M robados ese año.
- **WannaCry (2017):** explotó una vulnerabilidad conocida de Windows, se distribuyó por la red local; > 200K máquinas, costo estimado U\$D 4B.

Botnet: despliega **bots/agentes multiuso** (máquinas “zombi”) que quedan a la escucha de instrucciones de un **Command & Control (C&C)**. Las máquinas infectadas se usan para ataques distribuidos (DDoS), minería de cripto, accesos anónimos, distribuir spam, robar credenciales. La infraestructura C&C es un sistema **distribuido, descentralizado, sin único punto de falla**. **Srizbi (2008)**: la botnet más grande documentada, responsable de ~la mitad del spam mundial (60B mails/día).



Taxonomía de los tipos de malware vistos en clase. Virus/gusano y troyano describen cómo llega/se propaga; el resto describe el payload o la función.

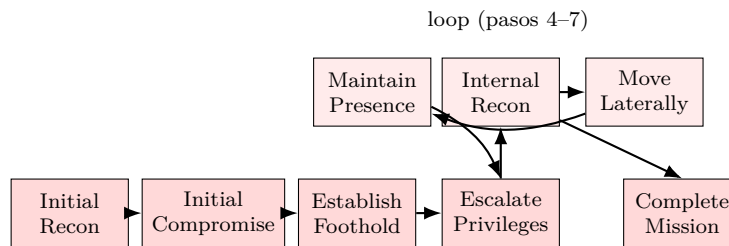
► En el parcial

Definiciones que se preguntan (verdadero/falso o emparejar):

- Virus/Worm → se *propaga*. Troyano → se *esconde* dentro de software benigno. Ni virus ni troyano dicen qué hace una vez adentro: eso es el **payload** (ransomware, C&C, spyware...).
- Rootkit → se *oculta* con contramedidas activas (no es un mecanismo de propagación, sino de ocultamiento/persistencia). Puede ocultar tanto virus como troyanos u otros programas. Se combinan: un virus se propaga, en cada máquina se enquista como rootkit y ejecuta un ransomware.

4.7.2. Ciclo de ataque del malware (Malware attack cycle)

1. **Reconocimiento inicial (Initial Recon):** se identifica a la víctima y se recopila información (puntos débiles, responsables clave, redes sociales de empleados; se establecen relaciones falsas).
2. **Compromiso inicial (Initial Compromise):** el malware se “*weaponiza*” (se empaqueta para ser ejecutado por la víctima) y se compromete una o más máquinas con un ataque dirigido. Puede durar varios días.
3. **Establecerse (Establish Foothold):** establece comunicación con su C&C, asegura persistencia y almacenamiento; baja y compone nuevas herramientas. Suele durar minutos tras la infección inicial.
4. **Escalar privilegios (Escalate Privileges):** eleva los privilegios obtenidos usando vulnerabilidades locales o remotas. Minutos.
5. **Reconocimiento interno (Internal Recon):** mapea otras máquinas y usuarios de interés; prueba comunicación/visibilidad; baja nuevos malwares.
6. **Movimiento lateral (Move Laterally):** explotación remota de vulnerabilidades para incrementar la base infectada.
7. **Mantener la presencia (Maintain Presence):** persiste en las nuevas máquinas. (Pasos 4-7 forman un **loop**.)
8. **Completar la misión (Complete Mission):** ejecuta su payload. Va exfiltrando o acumulando la información hasta enviarla en horarios de menor monitoreo; en ransomware, infecta la mayor cantidad posible de máquinas antes de activar el bloqueo.



Ciclo de ataque del malware: las primeras fases son secuenciales; escalada de privilegios, reconocimiento interno, movimiento lateral y mantener presencia forman un loop que expande la base infectada antes de completar la misión (payload).

Lo interesante: desde el reconocimiento inicial hasta el loop de escalamiento, el ciclo es **genérico** (independiente del payload final). Por eso existen **kits de desarrollo de malware** vendidos como producto en el mercado gris (aunque a veces son ellos mismos troyanos, y al ser genéricos tienen más chance de ser detectados).

△ Cuidado — error típico

Errores que penalizan en esta unidad:

- No reconocer que una **reducción de entropía es flujo de información** (caso del disipador / side-channel).
- Afirmar que el control de acceso (matriz de acceso) evita el flujo de información: **no lo evita**.
- Usar la fórmula de flujo equivocada: condicional $H(x_s | y_s)$ si y ya existía; absoluta $H(x_s)$ si y se crea durante la ejecución.

5. Seguridad en la Empresa

Esta unidad cambia el foco: deja la matemática y las vulnerabilidades de aplicación y mira cómo una **organización real** (corporación o PyME avanzada) se protege. El profe aclaró que **este tema no está en Bishop** (salvo lo que se pueda leer suelto); la clase está muy enfocada en el panorama profesional contemporáneo y en el estándar **NIST 800-207** (Zero Trust).

“Esto de clase que vamos a ver hoy no está en Bishop. Está más enfocado en el panorama contemporáneo, lo profesional, cómo se maneja esto en una organización.”

— dicho en clase

► En el parcial

La lectura recomendada de esta clase es **NIST 800-207 (capítulos 2 y 3)**, no Bishop. Toda la parte de Zero Trust sigue ese estándar.

5.1. Gobernanza (Governance)

Definición: Gobernanza de seguridad

Conjunto de **procesos, políticas, procedimientos y controles** que se implementan para gestionar y mitigar los riesgos de la organización, y el establecimiento de **responsabilidades y roles** para garantizar la protección de la información. Es como el “gobierno” interno: emite las reglas y se hace responsable de que se cumplan.

La gobernanza la lleva (generalmente) el equipo de seguridad. Su propósito es asegurar **confidencialidad, integridad y disponibilidad** de los datos, tanto de forma **proactiva** (poner barreras y controles) como **reactiva** (qué medidas tomo una vez atacado).

Objetivos principales:

- **Alinear la estrategia de seguridad con los objetivos del negocio.** No poner medidas que vayan en contra de cómo la empresa gana dinero.
- Reducir y mitigar los riesgos de la operación.
- Cumplir regulaciones y normativas aplicables (p.ej. empresas que cotizan en bolsa deben informar incidentes; datos personales mal protegidos ⇒ multas millonarias).
- **Crear cultura de seguridad** (campanas de concientización, de phishing, capacitación), porque el eslabón más débil es el usuario.

! Importante — remarcado en clase

El eslabón más débil es siempre el **usuario / empleado / cliente**. Por eso, si se ponen **demasiadas trabas**, el empleado busca atajos (se baja los datos a un CSV, los sube a Drive, etc.) y **termina atentando contra la seguridad**. Esto es el problema de la *aceptación psicológica*: la estrategia debe estar alineada con el negocio para no friccionar el trabajo.

5.2. Gestión de riesgos (Risk Management)

Históricamente la seguridad se gestionaba “como un riesgo más”, y ese modelo se mantiene. Es un **ciclo que nunca para** (los riesgos evolucionan, aparecen nuevos): **Identificar** → **Evaluar** (Assess) → **Mitigar** → **Monitorear** y vuelta a empezar.

5.2.1. Identificación del riesgo

Primero se identifican los **activos críticos**, que involucran: **Datos** (de empleados, clientes, comerciales, financieros, proveedores), **Sistemas, Infraestructura** (servidores, oficinas, data centers) y **Personas**.

Luego los **riesgos potenciales**: Malware, Ataques externos, Errores humanos, Sabotaje (interno) y Ambientales (huracán, incendio). El profe sumó los **problemas externos** (caída de una región de un proveedor cloud que voltea a muchas empresas a la vez).

5.2.2. Evaluación del riesgo

Se estima **probabilidad de ocurrencia** $\times$ **impacto en el negocio** (matriz probabilidad vs. impacto). Se clasifican en **Alto** / **Medio** / **Bajo** y se atacan primero los de mayor probabilidad y mayor impacto. Acá también se realizan evaluaciones externas (Red Team / PenTest).

5.2.3. Mitigación del riesgo — las 5 opciones

Definición: Tratamiento de un riesgo

Ante un riesgo identificado hay cinco maneras de manejarlo:

1. **Evitarlo**: eliminar la causa. Suele ser *costoso/complejo* (p.ej. resolver vulnerabilidades de un sistema legacy o migrarlo).
2. **Reducirlo** (lo más común): bajar el impacto sin eliminarlo (p.ej. sumar un factor extra de autenticación a una autenticación débil).
3. **Transferirlo**: delegarlo a un proveedor (p.ej. Cloudflare como proxy reverso). Si pasa algo, responde el proveedor.
4. **Compensarlo**: no se puede reducir, pero se contrarresta el impacto (p.ej. auditar/controlar continuamente una dependencia que no se puede quitar).
5. **Aceptarlo**: se entiende y se asume el riesgo (típico en riesgos de baja probabilidad). **Se debe documentar** la decisión.

► En el parcial

Distinguir **reducir** (baja la probabilidad/impacto del riesgo) de **compensar** (el riesgo sigue igual, pero se contrarresta su efecto). Un ejemplo de **aceptar**: muchas empresas tienen un **DRP** (Disaster Recovery Plan) documentado y exigido (p.ej. el Banco Central a las fintech), pero **no lo ejecutan** por costo $\Rightarrow$ aceptan el riesgo y lo dejan documentado.

5.2.4. Monitoreo

Es cíclico y permanente (la organización es un sistema vivo). Existen **frameworks** de apoyo: NIST (análogo del IRAM en Argentina) y los **CIS Controls** (18 controles) como guía de qué tener en cuenta.

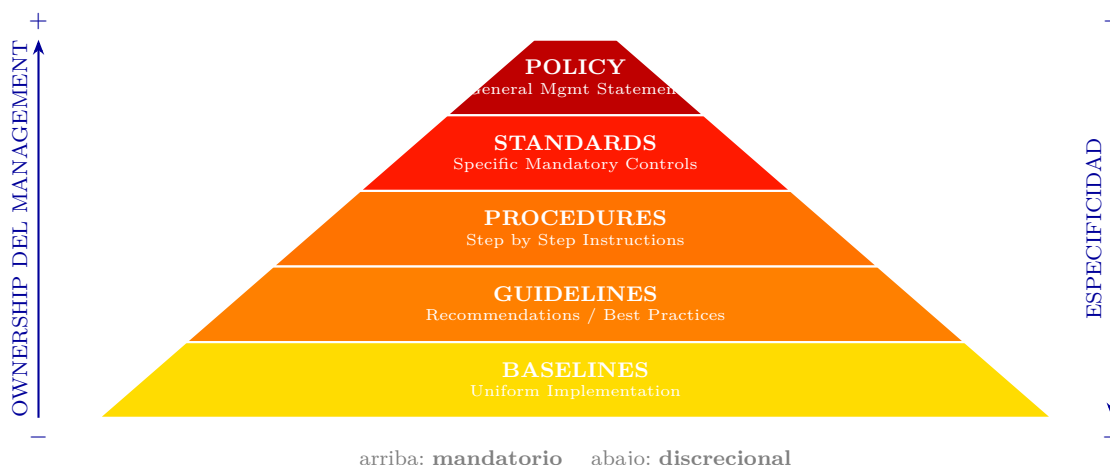
5.3. Políticas de seguridad

Definición: Política de seguridad

Documento **genérico y muy abarcativo** (“la Biblia” de la seguridad de la empresa); se confecciona cada cierto tiempo y lo aprueba la dirección (CISO/CEO). Define todo lo que compete a la seguridad, llegando incluso al plano legal (manejo de datos personales, estándares de industria).

De la política se va “bajando línea” con creciente especificidad (y decreciente *ownership* del management). Pirámide:

- **Policy** (declaración general del management) — mandatorio.
- **Standards** (controles específicos mandatorios).
- **Procedures** (instrucciones paso a paso).
- **Guidelines** (recomendaciones / mejores prácticas).
- **Baselines** (formas uniformes de implementación; cursos, capacitaciones, concientización).



Pirámide de políticas: hacia arriba crece el ownership del management y la obligatoriedad; hacia abajo crece la especificidad.

△ Cuidado — error típico

En la pirámide, hacia **arriba** crece el **ownership del management** y es **mandatorio**; hacia **abajo** crece la **especificidad** (y se vuelve más discrecional). No confundir el eje.

5.4. Estrategias: Defense-in-depth vs. Zero Trust

Surgieron estrategias genéricas para implementar las políticas. La clase destaca dos: **Defense-in-depth** (defensa en profundidad) y **Zero Trust Architecture (ZTA)**.

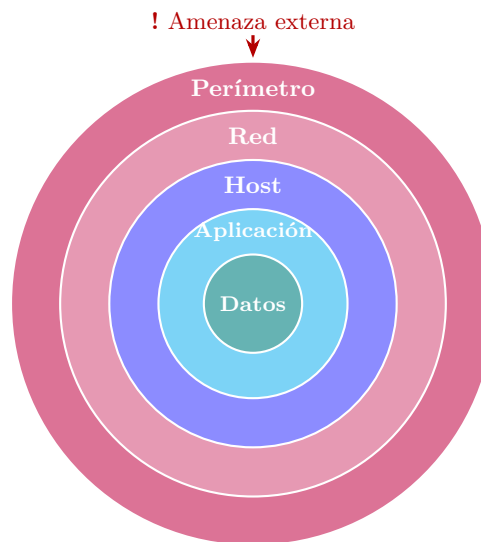
5.4.1. Defense-in-depth (defensa en profundidad)

Asume que **ningún control es perfecto**. Diseña **capas**, cada una pensada para contener a la siguiente (modelo “cebolla”), dando **redundancia**: si falla una capa, la de abajo contiene.

Las **5 capas** (de afuera hacia el dato):

1. **Perímetro (Perimeter)**: frontera con Internet. Firewall que deniega lo que viene de afuera y solo acepta cierta red o VPNs; honeypot; seguridad de mensajería (anti-virus/anti-malware); IDS/IPS de perímetro.
2. **Red (Network)**: dentro de la red corporativa. **Segmentación** (separar la red de operaciones de la red de servidores); NAC; Web Proxy / Content Filtering; DLP.
3. **Host / Endpoint**: *hardening* (“jardinizar”) del host — dejarlo inmutable, solo con el software permitido (idea tipo contenedor); Host IDS/IPS; antivirus.
4. **Aplicación**: buen diseño y buenas prácticas para no meter vulnerabilidades; **WAF**; monitoreo/escaneo de BD.
5. **Datos**: lo que más importa. Clasificación, cifrado, gestión de identidades y accesos (IAM), DLP, PKI.

Para llegar al dato, el atacante debe pasar perímetro → moverse lateralmente a la red de servidores (que no debería ser alcanzable) → vulnerar el host jardinizado → vulnerar la aplicación.



Defense-in-depth (modelo “cebolla”): cada capa contiene a la siguiente. El atacante debe atravesar Perímetro → Red → Host → Aplicación para llegar al Dato.

Principios.

- **Diversificación**: distintas tecnologías/métodos ⇒ una sola vulnerabilidad no atraviesa todo.
- **Redundancia**: cada capa contiene a la siguiente.
- **Segmentación**: dividir la red en segmentos controlados (se puede llegar al extremo de que una aplicación solo “vea” a la aplicación con la que necesita hablar).
- **Monitoreo**: visibilidad de todas las capas y **correlación de eventos** (si no se monitorea, nada de lo anterior importa).

Problemas.

- **Altos costos**: de tecnología (muchos proveedores), de capacitación/operación y de monitoreo (se recolectan muchos datos pero se les saca poco valor).
- **No resuelve bien los ataques internos** (si alguien ya está adentro, es difícil detectarlo; muchas herramientas no son bidireccionales).

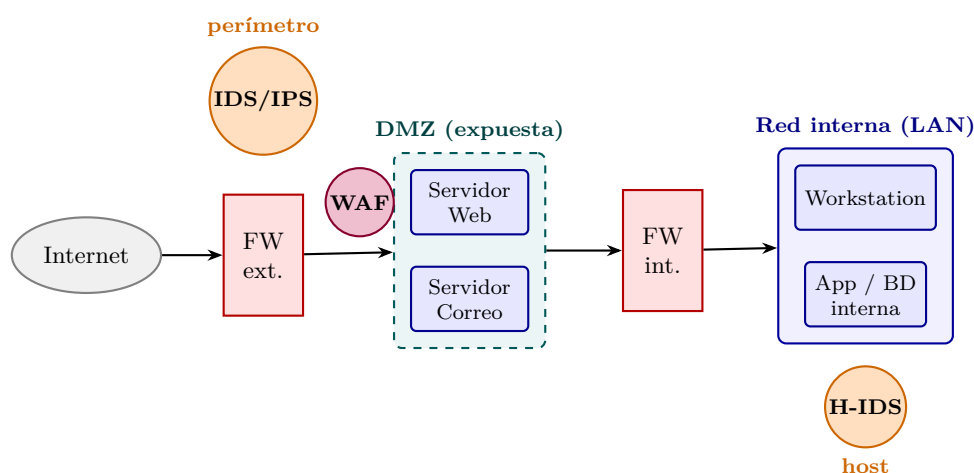
- **Difícil en infraestructura distribuida:** pensado para *on-premise*/data center propio; hoy hay data centers híbridos, multi-nube y empresas *cloud native*.

△ Cuidado — error típico

Firewalls en esta unidad (según lo visto en clase): el firewall de **perímetro** bloquea/filtra tráfico de red; el **WAF** (**Web Application Firewall**) aparece en la capa de **aplicación** y es “un firewall pero para el tráfico de una aplicación web”. En clase se mencionaron solo estos dos (firewall de perímetro y WAF).

! Importante — remarcado en clase

DMZ (zona desmilitarizada). En el diagrama de Defense-in-depth aparecen las *Secure DMZs* dentro de la seguridad de **perímetro/red**: es la zona expuesta hacia la red externa.



Topología con DMZ. El **firewall externo** expone solo la DMZ (web, correo); el **firewall interno** protege la LAN. El **IDS/IPS** se ubica en el perímetro y también en el host (Host IDS); el **WAF** se coloca delante de la aplicación web.

IDS / IPS.

Definición: IDS / IPS

Sistemas de **detección** (IDS, *Intrusion Detection System*) y de **prevención** (IPS, *Intrusion Prevention System*) de intrusiones. Funcionan “tipo antivirus”: detectan patrones a través de **firmas** o **anomalías** y determinan si **bloquear** (prevenir), **alertar** o tomar alguna acción.

► En el parcial

La diferencia clave: **IDS detecta** (alerta), **IPS previene** (bloquea). En el diagrama de Defense-in-depth aparecen tanto en el **perímetro** como en el **host** (Host IDS/IPS).

Otros componentes vistos en las capas.

- **Honeypot:** red “boba” a donde se deriva al atacante detectado en el perímetro para mantenerlo entretenido y que no llegue a la red que importa.
- **NAC (Network Access Control):** define si un dispositivo se puede conectar o no (por IP, MAC, firmware, credenciales); similar a limitar por MAC en un router.

- **Web Proxy / Content Filtering** (p.ej. Fortinet): bloquea/filtra tráfico (que la gente no entre a sitios maliciosos o no laborales).
- **DLP (Data Loss Prevention)**: evita la **exfiltración** de datos hacia afuera (escanea por datos de tarjeta, personales, etc.).

5.4.2. Zero Trust Architecture (ZTA)

Definición: Zero Trust Architecture (NIST 800-207)

Modelo de seguridad que asume que las amenazas **internas y externas están presentes en la red en todo momento**. Objetivo: proteger integridad, confidencialidad y disponibilidad de datos y recursos. Principios: **Nunca confíes (Never Trust)**, **Siempre verifica (Always Verify)**, **Asume que has sido atacado (Assume Breach)**.

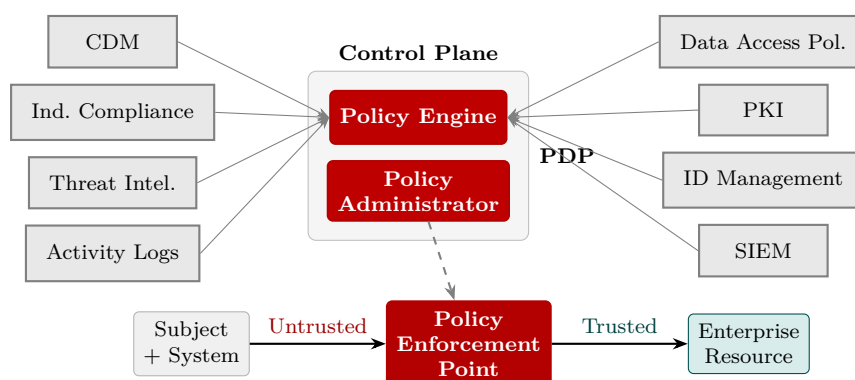
ZTA nace de constatar que **ninguna red es segura**: la workstation de un empleado (en la oficina o por VPN) puede tener malware sin que lo sepa, o puede haber un intruso físico. Por eso **cualquier red interna pasa a considerarse tan peligrosa como Internet**, y los controles que se aplicaban “afuera” ahora se aplican también “adentro”.

! Importante — remarcado en clase

La **intranet ya no es de confianza**. Antes era “buena práctica” relajar controles dentro de la red corporativa (para no friccionar al empleado); ZTA revierte eso: **cada componente/aplicación debe forzar la verificación**. En la práctica se relaja con **federación de identidad / SSO** (el empleado loguea una vez, p.ej. contra el directorio, y cada aplicación hereda su identidad, roles y permisos sin pedir login nuevo, pero *sí* aplicando los controles).

Componentes (NIST 800-207). El núcleo es el **Policy Decision Point (PDP) = Policy Engine + Policy Administrator**, y el **Policy Enforcement Point (PEP)** que separa la zona *Untrusted* (Subject/System) de la *Trusted* (Enterprise Resource). Lo alimentan:

- **CDM (Continuous Diagnostics and Mitigation)**: informa al motor de políticas sobre el activo que pide acceso (¿antivirus actualizado?, ¿últimos updates de seguridad?); puede desactivar el recurso si no cumple.
- **Industry Compliance**: garantiza el cumplimiento de regímenes normativos — **HIPAA, Banco Central, SOX, PCI**.
- **Threat Intelligence**: fuentes internas/externas sobre CVEs, firmas de ataques, IPs comprometidas, botnets (p.ej. VirusTotal).
- **Activity Logs**: registros de actividad de red y sistemas (datos crudos, en tiempo casi real).
- **SIEM (Security Information and Event Management)**: recolecta y **correlaciona** eventos para detectar anomalías y generar alertas.
- **ID Management**: crea/almacena/gestiona identidades de los usuarios (una persona puede tener varios usuarios), con sus roles y permisos.
- **PKI**: genera y registra los certificados de sujetos, recursos, servicios y aplicaciones.
- **Data Access Policy**: reglas/políticas de acceso a los recursos; punto de partida para autorizar (codificadas o generadas dinámicamente por el PE).



*Zero Trust (NIST 800-207): el **PDP** (Policy Engine + Policy Administrator) decide; el **PEP** aplica y separa la zona Untrusted (Subject) de la Trusted (Enterprise Resource), con verificación continua alimentada por CDM, Compliance, Threat Intel, Logs, PKI, ID Mgmt, SIEM y Data Access Policy.*

△ Cuidado — error típico

ZTA “parece la panacea” pero es **muy fácil de implementar mal**: o se implementa de forma deficiente, o se implementa y **no se controla / no se hace evolucionar**. Tener ZTA “puesto” no protege si no se monitorea y audita. Si una empresa arranca **de cero**, conviene ir directo a ZTA (sin el sesgo/lastre del esquema viejo).

5.5. Privileged Access Management (PAM)

Definición: PAM

Práctica para **proteger y gestionar cuentas privilegiadas** (acceso a sistemas críticos). Estos accesos son sensibles porque pueden conceder capacidades amplias, modificar configuraciones de seguridad, acceder a datos confidenciales y supervisar otras cuentas. Son lo que más buscan los atacantes.

Prácticas/herramientas:

1. **Inventario** de cuentas privilegiadas (visibilidad y control).
2. **Privilegio mínimo**: la cuenta privilegiada **no** es la cuenta de trabajo de la persona; es una cuenta separada (que puede transferirse al cambiar de rol).
3. **MFA** (autenticación multifactor) para garantizar acceso solo de autorizados y el **no repudio**.
4. **Registro y monitoreo** de toda la actividad (logs en servidor externo para reconstruir qué pasó).
5. **Rotación de contraseñas** (regular y automática; y reválida al transferir la cuenta).

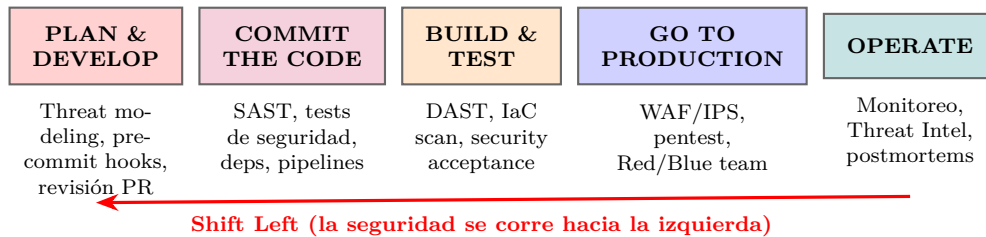
Ciclo de vida PAM: Define → Discover → Manage & Protect → Monitor → Detect Abnormal Usage → Respond to Incidents → Review & Audit.

5.6. Prevención de vulnerabilidades: DevSecOps

Definición: DevSecOps

Metodología que mete la **seguridad en cada paso** del proceso de llevar algo a producción, integrándola dentro de los equipos de desarrollo. Es la evolución de **DevOps** (que unió Dev y Ops bajo un mismo objetivo: “*you build it, you run it*”) sumándole el equipo de seguridad.

El problema histórico: Dev quiere sacar features rápido; Seguridad quiere frenar todo para revisar. Hoy la mayoría aplica seguridad recién en la **última etapa** (operación). DevSecOps propone el **Shift Left**: meter la seguridad lo más a la izquierda posible (planificación/desarrollo).



Pipeline DevSecOps: la seguridad se integra en cada etapa (plan/develop → commit → build/test → production → operate). El Shift Left empuja los controles lo más temprano posible.

“Cuanto más temprano detectamos un problema, menos difícil es resolverlo.”

— dicho en clase

Seguridad por etapa (Shift Left):

- **Plan & Develop: Threat modeling** (evaluar amenazas de una solución nueva antes de desarrollarla), plugins de seguridad en el IDE, **pre-commit hooks** (que no se expongan secretos), buenas prácticas, revisión de PRs.
- **Commit the code / PR: SAST** (análisis estático, p.ej. SonarQube), unit/functional tests de seguridad (foco en **CWE**), control de vulnerabilidades en dependencias, **pipelines seguros** (protección de secretos).
- **Build & Test: DAST** (análisis dinámico en sandbox), validación de configuración cloud / IaC, infrastructure scanning, security acceptance testing.
- **Go to Production:** evaluación de la app como un todo incluyendo el ambiente de despliegue (Firewalls, WAF, IPS), controles sobre artefactos y secretos, **pentests externos, Red/Blue team**.
- **Operate:** monitoreo continuo de métricas y logs, integración con Threat Intelligence, blind pentest, simulación de incidentes, respuesta a incidentes, *blameless postmortems*.

Testeos de seguridad en el código.

- **SAST** (Static): analiza el código *sin ejecutarlo*; marca malas prácticas, code smells y problemas de seguridad.
- **DAST** (Dynamic): evalúa la app **en ejecución**, interactuando como un usuario externo a través de las interfaces estándar (APIs, endpoints); busca fallas explotables en red (validación de entrada, autenticación). Automatiza pruebas repetibles (escanear puertos, inputs inválidos, etc.).

- **SCA** (Software Composition Analysis): analiza **dependencias y librerías** para identificar vulnerabilidades (incluso **transitivas**), problemas de licencia y otros riesgos. Debe extenderse a servicios SaaS consumidos.

! Importante — remarcado en clase

Caso real mencionado: **Log4Shell** (Log4j, 2021). Una vulnerabilidad crítica de *ejecución remota de código* en una librería de logging de Java usadísima. El problema: muchas empresas **no sabían si la usaban** (era una dependencia tan estándar que “te olvidás que está”). De ahí el resurgir del **SBOM**.

Definición: SBOM (Software Bill of Materials)

Inventario **formal y legible por máquina** de los componentes de software, dependencias, información sobre ellos y sus relaciones. Debe ser lo más completo posible (e indicar dónde no se pueden articular relaciones) e incluye software open-source y comercial. Permite saber qué componentes usa cada aplicación ante un incidente.

5.7. Protección de la nube

La configuración de la nube es **responsabilidad del cliente**. La **Infraestructura como Código (IaC)**, p.ej. Terraform, permite definir la configuración de forma reproducible; existen análisis estáticos que detectan configuraciones inseguras o ineficientes (puertos expuestos, redes abiertas).

Herramientas (productos caros, pensados para empresas con miles de servidores):

- **CSPM (Cloud Security Posture Management)**: busca problemas de configuración y permisos; analiza IaC *antes* de desplegar y revisa la config actual. Enfoque más **estático**.
- **CWPP (Cloud Workload Protection Platform)**: protege las aplicaciones corriendo en la nube (VMs, contenedores, serverless); analiza **comportamiento**, microsegmentación a nivel de contenedores, threat intelligence. Complementa a CSPM.
- **CIEM (Cloud Infrastructure Entitlement Management)**: foco en los **permisos** de los sujetos sobre recursos cloud; busca permisos excesivos comparando uso real vs. permisos otorgados; visualiza permisos entre cuentas.
- **CNAPP (Cloud Native Application Protection Platform)**: plataforma que integra lo anterior (visión seguridad proactiva + reactiva).

5.8. Evaluaciones de seguridad

“Tomamos todos los recaudos, todas las mejores prácticas... y en algún momento ese sistema va a fallar. No pregunten cuándo, no pregunten cómo, pero va a fallar.”

— dicho en clase

La seguridad nunca está asegurada $\Rightarrow$ hay que evaluar **constantemente**. Tipos de evaluación:

- **Evaluación de vulnerabilidades**: herramientas **automatizadas** que escanean en busca de vulnerabilidades **conocidas**.
- **Pentesting**: más exhaustivo, lo hacen **expertos** que intentan explotar vulnerabilidades de forma controlada.
- **Evaluación de riesgos**: identificar/evaluar/priorizar riesgos e impacto (la seguridad como una amenaza más).

- **Auditoría de seguridad:** revisión sistemática de políticas, procedimientos y controles (interna o externa).
- **Evaluación de cumplimiento:** obligatoria para regulaciones (**GDPR, HIPAA, PCI-DSS**, protección de datos personales).
- **Seguridad física:** control de acceso a instalaciones, cámaras, protección de archivos físicos y equipos.
- **Seguridad de aplicaciones:** revisión de código, pruebas de seguridad y análisis de arquitectura.

5.8.1. Penetration Test (PenTest)

Objetivo: agarrar el sistema **funcionando** y probarlo integralmente para identificar las vulnerabilidades que lo hacen pasar de un estado **seguro** a uno **inseguro**. Se hace con **alcance y tiempo definidos** (no se puede probar eternamente).

Tipos según visibilidad del evaluador:

- **Black Box / Blind:** el evaluador no tiene información previa (quizás solo el dominio/IP). Simula un atacante externo; favorece pensar “fuera de la caja” y descubrir vectores que el sesgo ocultaría. Razón práctica: no querer compartir info interna (motivos legales).
- **White Box:** acceso completo (código fuente, diagramas, credenciales). Evaluación exhaustiva de vulnerabilidades internas.
- **Gray Box:** intermedio (algún conocimiento, no todo). Simula un usuario con acceso limitado (empleado con privilegios restringidos).
- **Double Blind:** Black Box donde el **atacado tampoco sabe** del ejercicio; evalúa además las capacidades de monitoreo y respuesta. Es lo más parecido a la realidad (relacionado con Chaos Monkey).

Tipos según sistema evaluado: Red externa (servicios expuestos a Internet), Red interna (atacante interno: empleado descontento o malware en una workstation), Redes inalámbricas (NFC, Bluetooth, Wireless), Aplicaciones web (SQLi, XSS, etc.) e **Ingeniería social** (phishing, smishing).

Etapas (6 pasos): 1) Preparación (objetivo, alcance, tiempo, nivel de riesgo aceptado) → 2) Armado del plan de ataque (herramientas, ubicación) → 3) Identificar el equipo → 4) Definir el objetivo a alcanzar → 5) Ejecutar el ataque (termina al lograr el objetivo o al vencerse el tiempo) → 6) Integrar los hallazgos y reiniciar el ciclo (pivotear/escalar con lo obtenido).

5.8.2. Red / Blue / Purple Team

- **Red Team** (ofensivo): piensa y actúa como atacante malintencionado de forma controlada y ética — pentesting, ingeniería social, assessment de vulnerabilidades.
- **Blue Team** (defensivo): implementa y mantiene controles (firewalls, IDS, gestión de parches y configuraciones), monitorea y responde a incidentes.
- **Purple Team:** enfoque colaborativo que combina ambos para mejorar la postura de seguridad (fomenta comunicación en vez de aislamiento).

5.8.3. Breach and Attack Simulation (BAS)

Categoría de herramientas/técnicas que evalúan la efectividad de las defensas simulando las **tácticas, técnicas y procedimientos (TTP)** de un atacante real. Objetivo: identificar vulnerabilidades **y** medir la capacidad de la organización para **detectar y reaccionar**. Se ejecutan de forma **recurrente** en distintos puntos. Relacionado: *Breach Attack Simulation*/juegos de guerra (simular un escenario desfavorable puntual — “¿qué pasa si un atacante obtiene una ruta de admin del cloud?” — y analizar qué tácticas de defensa aplicar); es barato (juntar gente y plantear el escenario, a veces en ambiente controlado).

6. Pentesting

El **pentesting** (penetration testing o prueba de penetración) es una de las formas de corroborar el funcionamiento correcto de un sistema en cuanto a seguridad. Parte de la premisa de que *ya existen vulnerabilidades* y trata de probar que esas vulnerabilidades existen.

! Importante — remarcado en clase

Esta unidad corresponde al cierre del temario y **entra completa en el parcial**. El profe lo remarcó: “*Esto no es así un corolario, sino que va a entrar en el parcial.*”

— dicho en clase Lo más importante a saber de memoria es la **metodología de hipótesis de falla** (los pasos en orden) y la idea de que el pentesting **no garantiza seguridad**.

6.1. Formas de verificar un sistema: verificación formal vs. pentesting

Hay varias formas de corroborar el funcionamiento correcto de un sistema. Dos de ellas:

Definición: Verificación formal

Manera de comprobar **matemáticamente** que un sistema cumple con las restricciones que le imponemos. Esquema:

- **Precondiciones** → hipótesis sobre el estado del sistema (por ejemplo: el sistema está en un estado seguro).
- Se ejecutan las operaciones del sistema ($Op_1 \dots Op_n$) sobre un input dado.
- **Poscondiciones** → resultado de aplicar esas operaciones al input.
- **Requerimiento**: las poscondiciones cumplen las restricciones pedidas.

Sirve para probar matemáticamente que un sistema hace lo que dice hacer.

Características de la verificación formal según el profe:

- Es **súper compleja**: con un par de variables y operaciones ya es difícil de seguir; hay que mapear todas las posibilidades, precondiciones e interacciones entre componentes.
- Es **muy costosa y extensa**; a veces termina siendo **impráctica**: puede llevar más tiempo hacer la verificación formal que hacer el propio sistema.
- Peor aún cuando el sistema **evoluciona rápido**: antes de terminar la verificación ya quedó obsoleta y hay que hacer otra.
- Es realizable a nivel de funciones (como las viejas pruebas de escritorio: pruebo un input, otro input, qué pasa si le paso null), pero a nivel de un sistema completo se vuelve poco práctica.

Definición: Prueba de penetración (pentesting)

Prueba para verificar que un sistema cumple con ciertas restricciones.

- **Precondiciones** → hipótesis sobre el estado del sistema y la existencia de una vulnerabilidad.
- **Poscondiciones** → sistema comprometido.
- **Ejecución**: aplicar pruebas para intentar mover al sistema del estado inicial al estado

comprometido.

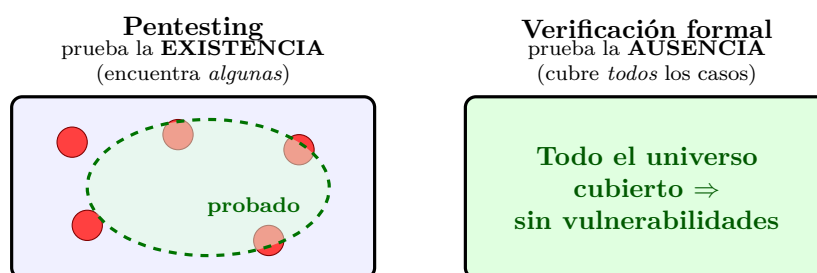
Es mucho más **económica, simple y eficiente** que la verificación formal, y es lo que hacen las empresas. Permite probar primero las áreas/componentes más críticos y después los menos críticos.

Similitudes y diferencias

- Ambas tratan de probar la **existencia** de vulnerabilidades.
- La **prueba de penetración NO prueba la ausencia** de vulnerabilidades.
- La **verificación formal SÍ prueba la ausencia**, pero para ello debe incluir **TODOS** los factores posibles (factores externos).
- En la práctica, la verificación formal prueba ausencia de vulnerabilidades en determinado algoritmo, programa o ambiente, **no en un sistema completo**. Se ignoran cosas como la **instalación** y el **uso**, el sistema operativo donde se instala, etc.

! Importante — remarcado en clase

El profe lo conecta con Dijkstra: el testing **puede mostrar la presencia** de bugs o vulnerabilidades, **pero no prueba su ausencia**. Esto va en total concordancia con el pentesting.



*Cada punto rojo es una vulnerabilidad real. El pentesting solo cubre una porción del espacio (zona punteada): si no encuentra nada, **no** prueba que no haya fallas fuera de lo probado. La verificación formal cubre todos los casos y **sí** prueba la ausencia, pero requiere incluir **TODOS** los factores y por eso en la práctica se acota a un algoritmo/programa/ambiente, no a un sistema completo. Por esto el pentesting **no garantiza la seguridad**.*

△ Cuidado — error típico

Que la poscondición sea “no encontré nada” / “no se dio la hipótesis” **no significa** que no haya alguna falla. Solo significa que el sistema **no está comprometido bajo las condiciones de esa prueba**. Puede haber otros vectores de ataque que no se probaron.

6.2. Objetivos y marco ético (ethical hacking)

El objetivo es **probar la eficacia de los controles de seguridad de un sistema**, intentando violar la **política de seguridad** existente. Requiere ejecutar técnicas similares a las de un atacante; por eso se lo llama **ethical hacking**.

- Generalmente hay un **equipo designado** que sabe de seguridad y conoce las implicancias. Hay que **simular ser un atacante** pero adoptando un comportamiento ético: no robar datos de clientes, no hacer compras a nombre de otra persona, etc.

- Es deseable que lo haga **alguien que no haya intervenido en la construcción** del sistema (análogo a QA: el desarrollador hace pruebas unitarias / de iteración / regresiones, y después un QA trata de romperlo con inputs inválidos, clics inesperados, etc.).
- Es **análogo a pruebas manuales** del sistema y **prueba al sistema como un todo**, no solo aspectos técnicos.
- **NO reemplaza un buen diseño e implementación**: estos tienen que estar de base y ser un requerimiento del sistema.

6.3. Metodología informal (preparación)

Vista como la etapa de preparación previa:

- **Determinar y cuantificar objetivos** (ej.: obtener información de clientes).
- Encontrar cierta cantidad de vulnerabilidades, o buscarlas durante un período de tiempo. El estudio se conduce **desde el punto de vista de un atacante**.
- Definir alcance, tiempo, propósito; armar el **plan de ataque**; elegir herramientas; seleccionar el equipo e identificar a quiénes van a participar y a estar al tanto (a veces **no se avisa**; lo del *lower line* / aviso a desarrolladores y equipos operativos se decide acá).
- Definir una condición de **objetivo** (éxito/no éxito). El ataque termina al lograr el objetivo o al vencerse el tiempo definido.
- **Estudiar y categorizar los hallazgos**: el éxito de una prueba de penetración proviene del **análisis de los hallazgos**. Es un proceso **cíclico**: los hallazgos se reincorporan al ciclo, permitiendo detectar problemas recurrentes y corregir sistemáticamente familias de problemas (una misma vulnerabilidad puede repetirse en varios componentes por usar la misma dependencia o forma de desarrollo).

6.4. Metodología de Hipótesis de Falla (LOS PASOS)

! Importante — remarcado en clase

“Esta metodología es importante, súper importante que la entiendan y la sepan; es la base de lo que hoy se usa para testear.”

— dicho en clase Es la base para equipos de Red Team, AppSec e InfraSec. Para sistemas medianos o grandes hoy se exige tener algún tipo de pentesting, sobre todo en los componentes *core*, donde se pide pasar el pentesting sin vulnerabilidades críticas o altas (definidas por los CVE; importancia de la organización **MITRE**).

Los **pasos en orden** (tal como aparecen en el PDF):

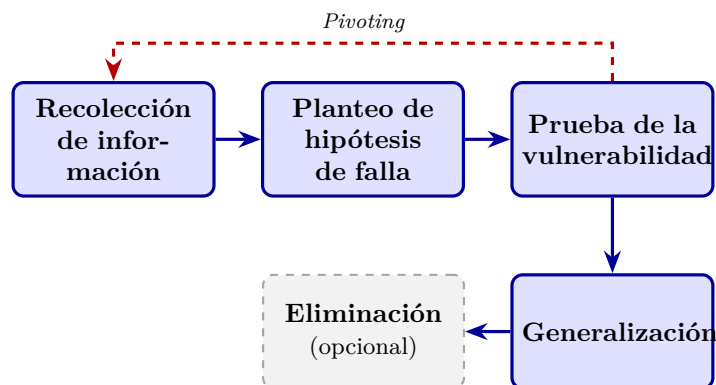
1. **Recolección de información** — para conocer el sistema y su funcionamiento.
2. **Hipótesis (Planteo de hipótesis de falla)** — asumir la existencia de vulnerabilidades concretas.
3. **Prueba** — probar las vulnerabilidades.
4. **Generalización** — generalizar patrones y hallar otras vulnerabilidades del mismo tipo.
5. **Eliminación (opcional)** — determinar los pasos necesarios para eliminarlas.

► En el parcial

! Foco de parcial (2025-2Q): los pasos del pentesting. El orden correcto incluye explícitamente el “**Planteo de hipótesis de falla**”. Secuencia tipo:

Recolección de info → Planteo de hipótesis de falla → Prueba → Generalización → Eliminación

El profe lo llama conocimiento “enciclopédico”: hay que saberlo de memoria. **Equivocar el orden o los pasos PENALIZA.** Notar que la **Eliminación es opcional** (solo se incluyen recomendaciones).



Metodología de hipótesis de falla: los pasos en orden (memorizar). La Eliminación es opcional (línea punteada). El Pivoting reinyecta nueva información a la fase de recolección, haciendo el proceso cíclico.

6.4.1. Paso 1: Recolección de información

- **Componer un modelo del sistema y sus partes.** Hay que conocer bien el sistema para saber “dónde poner el ojo”.
- **Buscar discrepancias:** cuando el sistema evoluciona y la documentación no refleja esa evolución (la doc dice que se comporta de una forma y ya no es así) → posible punto de entrada.
- **Revisar interfaces:** conexiones con sistemas externos, proveedores, otros sistemas, bases de datos.
- Buscar **vulnerabilidades conocidas genéricas** compatibles con la tecnología que usa el sistema.
- Apoyarse en la **documentación** (cuanta más, mejor); prestar especial atención a **secciones poco especificadas o ambiguas** (donde pudo tomarse una decisión cuestionable → otra puerta de entrada).
- Ver **manejo de privilegios y tipos de cuentas:** enumerar usuarios, buscar cuentas *super-admin* (las que más posibilidades dan, no están limitadas), cuentas **huérfanas** o con **exceso de permisos**.
- Conocer el **ambiente:** nombres de usuarios/servidores, en qué servidores corre, configuración y **estructura de red**.

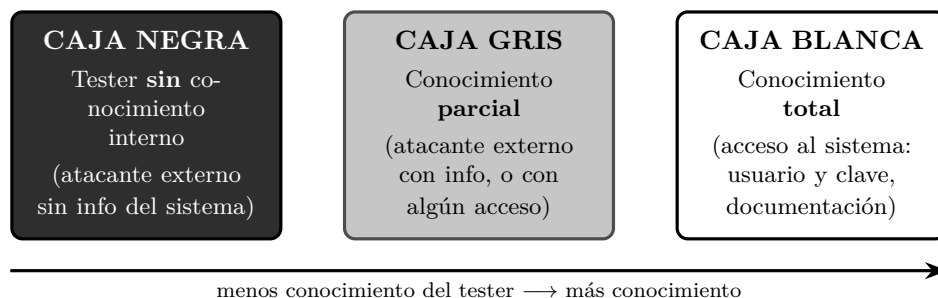
Punto de partida. Identifica la información inicial con la que cuenta el equipo (supuesto atacante). **Niveles incrementales:**

1. Atacante externo **sin** conocimiento del sistema.

2. Atacante externo **con** conocimiento del sistema.
3. Atacante **con acceso** al sistema (ya tiene usuario y contraseña).

Dependiendo del tipo de prueba, algunos niveles son irrelevantes: durante el diseño se ignora el primer nivel; en sistemas con registro abierto se ignora el segundo nivel.

Estos niveles de conocimiento inicial se corresponden con los tres tipos clásicos de prueba según cuánto sabe el tester del sistema:



Caja negra / gris / blanca según el conocimiento que tiene el tester del sistema. Se mapea con los niveles incrementales del punto de partida: externo sin info (negra), externo con info / acceso parcial (gris), acceso completo (blanca).

6.4.2. Paso 2: Hipótesis

Buscar posibles vulnerabilidades / combinaciones que expongan una vulnerabilidad:

- **Examinando políticas y procedimientos:** buscar inconsistencias, e inconsistencias entre políticas y mecanismos. Los procedimientos pueden **no cumplirse**.
- **Examinando implementaciones:** usar modelos de vulnerabilidades; revisar vulnerabilidades comunes/conocidas (*vulnerability scanning*, ej. **portmapper**); usar manuales (exceder límites, omitir pasos de las secuencias, etc.).
- **Examinando mecanismos:** pueden estar mal implementados, el ambiente donde corren puede introducir errores, o pueden no ser seguros.
- **Comparando con otros sistemas:** sistemas parecidos tienen problemas parecidos.

Áreas concretas que el profe menciona como candidatas de hipótesis:

- **Seguridad en APIs:** política de desarrollo de **microservicios** (más servicios = más “cajitas” atacables; si cada equipo no baja la misma política de seguridad, hay diseño menos robusto). **Software Supply Chain (API) Management:** hoy es **top 3** del OWASP Top Ten para apps web; abarca dependencias vulnerables, infiltración en flujos CI/CD con plugins/dependencias vulnerables, y ataques de confusión de paquetes (ej. casos de **npm**, donde se baja todo el árbol de dependencias transitivas).
- **RASP (Runtime Application Self Protection)** mal implementado: herramientas que controlan que los binarios del sistema no cambien durante la ejecución.
- **Verificar Federadores:** protocolos abiertos y antiguos (ej. OAuth para Google u otras integraciones) que las empresas implementan mal; vulnerabilidades conocidas sobre protocolos.
- **Software EOL (End Of Life):** fuera de soporte, sin actualizaciones de seguridad → “muy jugoso” a la hora de pentestear.

Resultado del paso 2: una lista de posibles vulnerabilidades que asumimos que tiene el sistema.

6.4.3. Paso 3: Prueba

- **Priorizar** la lista de posibles vulnerabilidades, según el objetivo de la prueba y, por lo general, según el **nivel de acceso requerido**.
- **Definir cómo probar** la existencia de la vulnerabilidad:
 - **Mejor manera:** analizar documentación o comportamiento.
 - **Otra manera:** intentar **explotarla** → último recurso, menos eficiente (hay que lograr el exploit como lo haría un atacante). **Excepción:** *Pivoting*.
- **Diseñar el test para que sea lo menos intrusivo posible:** algunas vulnerabilidades pueden **denegar el servicio**.
- **Proceso normal:** resguardar los datos de todo el sistema; documentar los requerimientos para detectar la vulnerabilidad; intentar detectarla.
- La prueba **DEBE ser REPETIBLE y CONCLUSIVA**: si funciona una sola vez y no se puede replicar, no concluye que haya una vulnerabilidad. Si yo lo puedo repetir, naturalmente lo puede repetir un atacante.

! Importante — remarcado en clase

Hoy por hoy se asume que **para demostrar una vulnerabilidad hay que tratar de explotarla** (las “PoC” del pentesting), siempre con cuidado, sobre todo en sistemas productivos: un exploit puede llevar el sistema a un estado inseguro, exponer datos, tirar abajo la aplicación (atentar contra Confidencialidad, Integridad o Disponibilidad — el “CID”). Ejemplos del profe: el caso **Chernóbil** (era una prueba controlada y terminó mal) y correr un **nmap** (escaneo/discovery de red) que tiró abajo la red de la oficina, afectando la disponibilidad.

△ Cuidado — error típico

En pentesting **NO siempre se intenta explotar** las vulnerabilidades encontradas; explotar es el *último recurso* y a veces alcanza con analizar documentación o comportamiento. (V/F, 2015.) En caja negra puede que sí se explote, pero el profe lo matiza: “intentar explotar es el último recurso, menos eficiente”.

Pivoting (excepción). Es probable que la existencia de una vulnerabilidad provea más información (ej. acceso al S.O.). En ese caso se **vuelve a la fase de recolección de información**. **Pivoting** es el uso de un **punto de acceso comprometido para continuar un ataque** (ej.: se compromete el firewall y desde allí se continúa atacando la red interna). Por eso se encadenan vulnerabilidades y se vuelve al ciclo.

6.4.4. Paso 4: Generalización

- A medida que las pruebas resultan exitosas, **emergen patrones**; se “conectan puntos”.
- A veces **dos vulnerabilidades combinadas** constituyen un problema grave. Ejemplo: cuenta de invitado habilitada permite conexión remota + buffer overflow local permite derechos de administrador ⇒ desde una cuenta de invitado se obtienen permisos de administrador.
- Es la **parte más importante** de la prueba: la “mano” del pentester es identificar cosas, combinarlas y producir el mayor impacto posible para dar *insight* al equipo/empresa que lo contrató.

6.4.5. Paso 5: Eliminación (opcional)

- Por lo general **solo se incluyen recomendaciones**, porque quien ejecuta la prueba no suele ser el diseñador/desarrollador (es el dueño del sistema quien decide qué hacer). A veces no se tiene la solución concreta (vulnerabilidades muy nuevas); a veces es tan simple como actualizar la versión de una dependencia, a veces es más abstracto (ej. “manejá bien la autorización”).
- Es importante que queden claros el **contexto, los detalles y el mecanismo de explotación** para: poder corregir el sistema; poder impedir/monitorear mientras tanto; verificar si fue explotado; y **reproducir la prueba** luego de corregir, para comprobar que ya no es vulnerable.
- Todo termina en un **reporte de pentesting** con las vulnerabilidades encontradas y las recomendaciones.

6.5. Herramientas y técnicas mencionadas

- **nmap**: programa de Linux que escanea interfaces y puertos, arma una topología de la red y hace un *discovery* (qué está abierto/cerrado, qué se conecta con qué). *Ojo*: mal usado puede tirar abajo la red.
- **Vulnerability scanning** (ej. portmapper) para vulnerabilidades conocidas.
- **Ingeniería social** (ver ejemplo de ataque externo).
- **CVE** (catálogo de vulnerabilidades conocidas, con su remediación) y la organización **MITRE**.
- **OWASP Top Ten** (referencia para apps web; Supply Chain como top 3).

6.6. Ejemplos

6.6.1. Ejemplo 1: Michigan Terminal System (MTS)

Sistema operativo que corre en **mainframes IBM 360/370**. Objetivo de la prueba: **obtener acceso a las estructuras de control** que no debería permitirse. Nivel inicial: **cuenta autorizada (nivel 3)**. Contaba con la **aprobación de los administradores**.

- **Paso 1 (Recolección)**: al correr un programa, la memoria se divide en segmentos. Segmentos **0–4**: supervisor, programas de sistema y estado, protegidos por hardware. Segmento **5**: área de trabajo del sistema e información del proceso (incluido el nivel de privilegio); el proceso **no** puede alterarlo. Segmentos **6+**: información del proceso, que el proceso modifica libremente. El segmento 5 está protegido por memoria virtual: en **modo sistema (kernel)** el proceso puede acceder/modificar y ejecutar funciones privilegiadas; en **modo usuario** el segmento 5 no se mapea. Un proceso corre en modo usuario y hace *system calls* (que corren en modo sistema). En un system call: se revisan los parámetros (especialmente que los punteros pertenezcan a zonas accesibles por el proceso); los parámetros se construyen como una **lista de direcciones** en el segmento de usuario; la lista se pasa como dirección en un registro.
- **Paso 2 (Hipótesis)**: ¿qué ocurriría si la dirección de un parámetro **apunta a la lista misma de parámetros**? Sin controles extra, un system call podría escribir la dirección del parámetro **después de haber pasado la validación**. Si se puede controlar qué valor escribir, técnicamente se podría leer/escribir cualquier zona del **segmento 5**.

- **Paso 3 (Prueba):** buscar un system call que use la convención estudiada, tenga al menos dos parámetros y altere uno. Se eligió `line input` (lee una línea de texto y retorna número de línea y longitud de línea). *Setup:* hacer que la dirección donde se almacena el número de línea sea la de la longitud de línea. *Ejecución:* el system call valida los parámetros (todos pertenecen al segmento del proceso); ejecuta `read input line`; la longitud de línea ingresada es el valor a escribir; el número de línea se guarda según la lista de parámetros (haciendo que sea una dirección del segmento 5, reescribiendo la dirección del otro parámetro), y así se escribe el valor en el segmento 5.
- **Paso 4 (Generalización):** no se puede escribir en los segmentos 0–4 (protegidos por hardware), pero en el segmento 5 el nivel de privilegio determina si pueden hacerse **llamadas de nivel supervisor** (en lugar de system calls), y una función de nivel supervisor **apaga la protección por hardware**. **Conclusión:** la vulnerabilidad permite a un atacante modificar cualquier dirección de cualquier segmento, controlando completamente la computadora.

6.6.2. Ejemplo 2: Ataque externo (ingeniería social)

Objetivo: determinar si las medidas de seguridad de una empresa eran efectivas para evitar que un atacante ingrese a un sistema. El test se enfoca en **políticas y procedimientos, tanto técnicos como no técnicos**, muy apalancado en **ingeniería social** (es más fácil vulnerar a las personas que a los sistemas).

- **Paso 1 (Recolección):** búsqueda en internet de nombres de empleados y directores y teléfono de la sucursal local; del **reporte anual público** (la empresa cotizaba en bolsa) reconstruyeron el organigrama, nombres de proyectos y personas. El tester se hizo pasar por nuevo empleado; aprendió que para enviar un documento se necesitaba **número de empleado y centro de costos**. Llamó a la secretaria del director: haciéndose pasar por empleado consiguió el número de empleado del director, y haciéndose pasar por auditor consiguió el centro de costos. Con eso solicitó enviar el listado a una consultora externa.
- **Paso 2 (Hipótesis):** los empleados nuevos no conocen todos los procesos y controles → es posible que un nuevo empleado brinde información sensible.
- **Paso 3 (Prueba):** el tester llamó a **RRHH** impersonando a la secretaria de un director (se quejó de que no le habían informado los nuevos ingresos y los pidió), obteniendo los nombres de los nuevos empleados. Luego llamó a esos nuevos empleados haciéndose pasar por **operador del centro de cómputos** y les brindó un “entrenamiento de seguridad” por teléfono, durante el cual obtuvo: tipos de sistemas usados, número de usuarios, **logins y claves** (incluso induciendo el cambio de una contraseña para conocerla). Se probó que los nuevos empleados no estaban al tanto de los procesos/controles.

△ Cuidado — error típico

Aunque el objetivo era otro, si se encuentra una falla distinta hay que **informarla igual**. Ejemplo del profe: el objetivo era probar la autenticación, pero se encontró que yendo a una URL del tipo `/info.txt` desde el browser de cualquier usuario quedaban expuestos datos de tarjetas → se debe reportar más allá del objetivo.

6.7. Limitaciones: el pentesting NO garantiza la seguridad

► En el parcial

! GANCHO/TRAMPA de parcial. El pentesting es una buena estrategia pero **NO garantiza la seguridad**. “*Garantizar la seguridad es algo que no se puede hacer jamás.*”

— dicho en clase

- Si en un ejercicio alguien afirma que algo “**garantiza la seguridad**”, es incorrecto y **hay penalización si se les escapa**.
- La alternativa con **más garantía** (aunque *tampoco* total) son las **pruebas / verificación formal**, hoy un área hipercentral por el código generado por **LLMs/agentes**.
- No confundir: la verificación formal *prueba la ausencia* de vulnerabilidades pero solo en un algoritmo/programa/ambiente acotado y debiendo incluir **TODOS** los factores; el pentesting *nunca* prueba ausencia.

Razones por las que el pentesting no garantiza seguridad (“Para discutir”):

- **No reemplaza** un buen diseño, especificación, implementación ni las pruebas (unitarias, de interacción, regresiones). Es una técnica importante para probar el sistema **luego de instalado**; idealmente no sería necesario, en la práctica sí.
- Encuentra **problemas introducidos por la interacción del sistema con los usuarios y el ambiente**, que suelen quedar fuera del análisis y pruebas normales.
- **No es sistémico / no es sistemático**: la calidad depende de la **capacidad de los testers para formular hipótesis** (metodología de hipótesis de falla). No provee una forma sistemática de revisar un sistema.
- **Cada test es un mundo aparte**: los resultados de un test sirven solo marginalmente para otros. Hay herramientas que automatizan *algunos* aspectos, pero no todos.
- Un pentesting **vale para ese sistema en ese momento**: si después cambia la API o el comportamiento (v2 → v3), la mayoría de las pruebas quedan obsoletas.

! Importante — remarcado en clase

Conclusión del profe: siempre hay que tener buen diseño, respetar los principios de seguridad y las especificaciones, y hacer buena cobertura de testing; el pentesting se suma *después* para comprobar que el diseño es bueno. “*Siempre algo aparece.*”

— dicho en clase

6.8. Lectura recomendada

- Matt Bishop, *Computer Security: Art and Science*, Capítulo 23 (1–2).
- **OSSTMM** — Open Source Security Testing Methodology Manual (<http://www.isecom.org/osstmm/>).

7. Protección de Datos

Esta unidad se mira desde la perspectiva del **CISO** (*Chief Information Security Officer*), el responsable de velar por la seguridad de la información de la empresa (con implicancias legales si hay un incidente severo). Se aplican todos los principios ya vistos para proteger la **CIA** (Confidencialidad, Integridad y Disponibilidad) de los datos.

7.1. El problema y el ciclo de protección

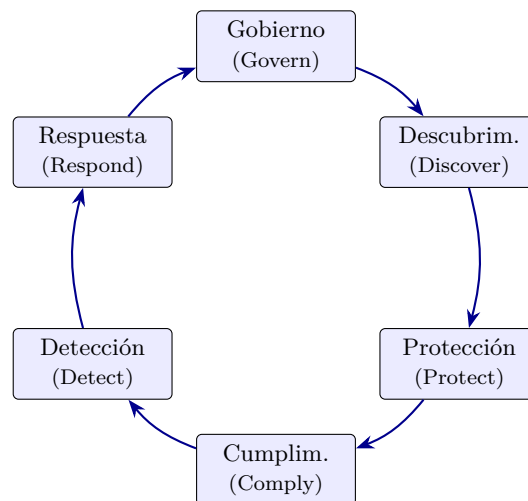
El problema motivador: en una empresa se define una política de protección de datos personales (ej. “un empleado no puede ver los sueldos de otro”). Los **controles de acceso son parte de la solución pero no son suficientes**: un mismo dato (un sueldo) fluye por RR.HH. → App → Excel → Email → nube, con backups en el medio. Hay que protegerlo en todo su recorrido.

Definición: Dato personal

Todo lo que permite **identificar a una persona** solamente por ese mismo dato: nombre y apellido, dirección de mail, número de documento, domicilio. Dentro de la organización hay además datos **sensibles** (ej. el sueldo) que no se quieren divulgar.

Ciclo de protección de la información (es un ciclo, vuelve a empezar):

1. **Gobierno (Govern)**: gestión, toma de decisiones.
2. **Descubrimiento (Discover)**: qué datos hay en el universo de la organización.
3. **Protección (Protect)**: cómo y qué datos protegemos (no todos son sensibles).
4. **Cumplimiento (Comply)**: asegurar que la protección efectivamente ocurre.
5. **Detección (Detect)**: detectar filtraciones / *breaches*.
6. **Respuesta (Respond)**: responder ante filtraciones y evitar que se repitan.

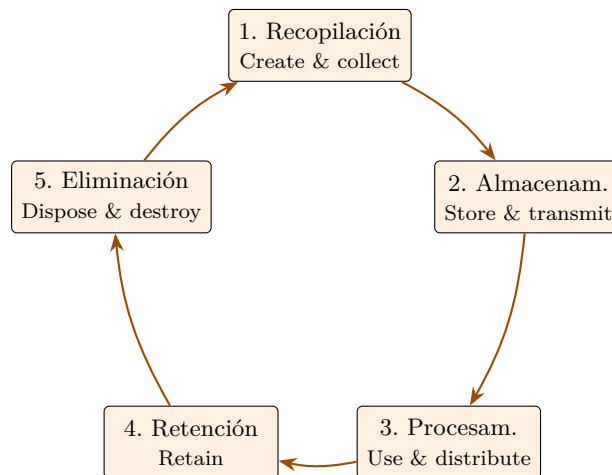


Ciclo de la protección de la información: es un ciclo, al terminar Respond se vuelve a Govern.

7.1.1. Data Governance

Son **todas las tareas para asegurar que los datos estén seguros, privados, íntegros y disponibles**. Incluye las medidas que toman las **personas**, los **procesos** que siguen y la **tecnología** que los respalda durante el **ciclo de vida de los datos**. Define los roles dentro de la organización. Establece estándares internos (**políticas de datos**) sobre: recopilación, almacenamiento, procesamiento, retención y eliminación.

Data Life Cycle (PwC): (1) Create & collect → (2) Store & transmit → (3) Use & distribute → (4) Retain → (5) Dispose and destroy (al vencer el período de retención).

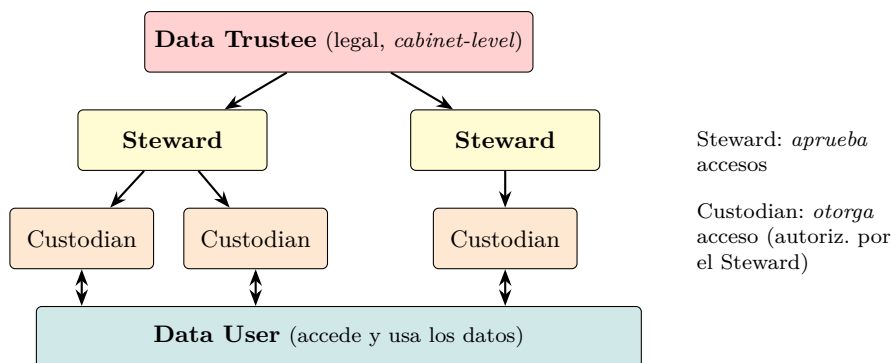


Ciclo de vida del dato (PwC): la eliminación ocurre al vencer el período de retención y el ciclo se reinicia.

7.1.2. Data Roles (jerarquía)

- **Data Trustee:** posición estratégica/*cabinet-level*, responsable **legalmente** de los datos de su unidad.
- **Data Steward:** responsable interno que **aprueba accesos** a los datos dentro de su dominio.
- **Data Custodian:** autorizado por un Steward a **otorgar acceso** a los usuarios dentro del dominio del Steward.
- **Data User:** quien efectivamente **accede** y usa los datos.

A medida que se baja en la jerarquía, el dominio de datos manejado es cada vez más chico.



Roles del dato: al bajar en la jerarquía el dominio de datos manejado es cada vez más chico.

7.2. Marco legal y regulaciones

! Importante — remarcado en clase

La distinción más preguntable: la **Ley argentina 25.326** regula las bases de datos (públicas y privadas). El **Derecho al Olvido** lo regula el **GDPR europeo**, NO la ley argentina.

7.2.1. Regulación argentina: Ley 25.326

Definición: Ley 25.326 – Protección de Datos Personales

Ley argentina (de los años 80–90). Aplica a todo sistema que opera en Argentina.

- **Art. 1 (Objeto):** protección integral de los datos personales asentados en archivos, registros, bancos de datos u otros medios técnicos de tratamiento, **sean públicos**

o **privados**, para garantizar el derecho al honor y a la intimidad de las personas (conforme art. 43, párr. 3 de la Constitución).

- **Art. 9 (Seguridad de los datos):** el responsable/usuario del archivo debe adoptar las medidas técnicas y organizativas necesarias para garantizar seguridad y confidencialidad, evitando adulteración, pérdida, consulta o tratamiento no autorizado, y permitiendo detectar desviaciones. Queda **prohibido** registrar datos en bancos que no reúnan condiciones técnicas de integridad y seguridad.

! Importante — remarcado en clase

La ley **no especifica una tecnología** concreta. Esto es a propósito (“para bien”): la generalidad hace que la ley **siga siendo aplicable hoy** pese a ser de hace décadas. Implica también que las personas pueden **ver, modificar e incluso pedir el borrado** de sus datos (hábeas data).

Hábeas data vs. hábeas corpus. El **hábeas data** es el derecho por el cual sos dueño de tus datos: podés pedir verlos, rectificarlos o que se borren (a diferencia del *hábeas corpus*, que protege la libertad física de la persona). Empresas como Facebook/Twitter permiten descargar todo tu historial y borrar la cuenta.

7.2.2. Regulaciones internacionales

Norma	Descripción
HIPAA (1996)	<i>Health Insurance Portability and Accountability Act</i> . Protección de datos de salud (PHI/PII). Surge la anonimización de datos (operar sobre historias clínicas sin saber de quién son).
GDPR (2018)	<i>General Data Protection Regulation</i> (Unión Europea). La más conocida. Aplica aunque el sistema esté en Argentina si hay usuarios/infraestructura en la UE. Pide consentimiento explícito (de ahí los <i>banners</i> de cookies). Regula el Derecho al Olvido .
PCI-DSS	<i>Payment Card Industry Data Security Standard</i> . Tarjetas de crédito. Acotó la cantidad de datos a proteger (número, titular, vencimiento, código de seguridad/CVV, banda magnética). Introdujo la tokenización .
SOX (2002)	<i>Sarbanes-Oxley Act</i> (ley federal de EE.UU.). Empresas que cotizan en bolsa; exige transparencia. Retención de auditoría 7 años .
FFIEC	<i>Assessment Tool</i> (framework, hoy en desuso).
FDA Cybersecurity	Ciberseguridad en dispositivos médicos.

! Importante — remarcado en clase

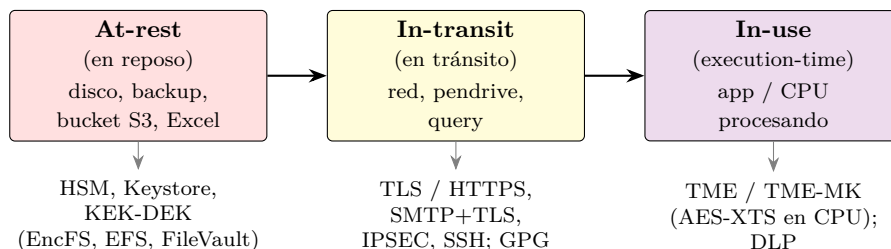
El gran aporte de **PCI-DSS** (“Blue Print”): a la hora de proteger datos hay que **acotar la cantidad de datos en base a su criticidad**. No proteger todo por igual, sino concentrarse en lo crítico.

Tokenización (PCI-DSS). Los datos críticos de la tarjeta se guardan “bajo 7 llaves” en una base recontra protegida; el sistema opera con un **token** que apunta a la tarjeta. Si se filtra el token, no tiene valor por sí solo: habría que ir a buscar los datos reales a la base ultra-protegida.

7.3. Estados del dato y encriptación

Un mismo dato puede estar en distintos **estados**; las regulaciones definen lineamientos para cada uno:

- **En reposo (at-rest)**: el dato está quieto, no se procesa ni se mueve (base de datos, backup, Excel, bucket S3).
- **En tránsito / movimiento (in-motion / in-transit)**: se mueve de una ubicación o estado a otro (a través de la red, un pendrive, una query).
- **En uso (in-use)**: está siendo procesado/usado por una aplicación o CPU.
- **En memoria**: caso particular; puede verse como *at-rest* en memoria esperando a la CPU, o *in-transit* entre el bus de I/O y la CPU.



Mismo dato en distintos estados y la técnica de cifrado en cada uno.

7.3.1. Enforcement (Compiler-time)

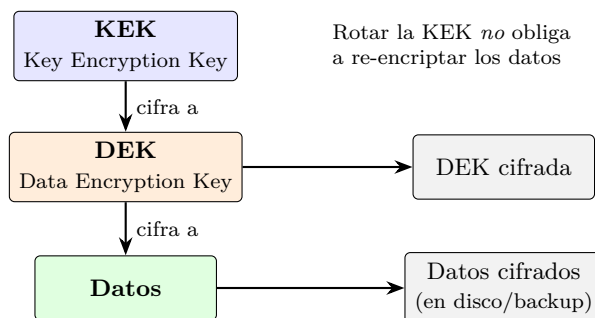
Asegura que el código cumpla políticas de seguridad **antes** de ejecutarse, vía **análisis estático de código** en la compilación (ej. detectar que se imprimen contraseñas/secretos). El compilador puede asegurar que datos confidenciales no se filtren a zonas de memoria de menor seguridad. Ejemplo: `GuardedString` de Java (guarda el string **encriptado** en memoria; un `String` clásico se ve en texto plano inspeccionando el proceso).

7.3.2. Encriptación at-rest

Objetivo: proteger datos inactivos (disco, pendrive, etc.). Mecanismos:

- **HSM** (*Hardware Security Module*, FIPS 1–4): hardware **anti-tampering** que guarda secretos (ej. claves simétricas) y ofrece las operaciones de cifrado/descifrado **sin exponer la clave**.
- **Keystore**: software que guarda todas las claves encriptadas con una clave **máster**.
- **Esquema KEK-DEK (encriptación por capas)**:
 - **DEK** = *Data Encryption Key* (cifra los datos).
 - **KEK** = *Key Encryption Key* (cifra la DEK).

Más seguro y flexible: se puede **rotar la KEK** sin re-encriptar los datos (la DEK no cambia). Más difícil de administrar (segunda capa). Ejemplos: EncFS/Loop-AES, EFS (Windows), FileVault (Mac OS X). Problemas asociados: *Data Remanence* y *Secure Deletion*.



Jerarquía de claves (envelope encryption): la KEK cifra la DEK y la DEK cifra los datos. Se almacenan la DEK cifrada y los datos cifrados.

At-rest en la nube. La nube es infraestructura de terceros. Espectro de opciones (de mayor control/menor costo a mayor eficiencia operativa/mayor costo):

Opción	Quién controla las claves
Own Encryption	Encriptación del lado del proceso antes de almacenar. Independiente del proveedor.
Hold Your Own Keys (HYOK)	El cliente gestiona las claves, posee las privadas y decide la rotación.
Bring Your Own Keys (BYOK)	El cliente trae sus claves.
Customer Managed Keys	El cliente decide rotación y puede encriptar lo que desee, pero no accede a las claves privadas.
Service Managed Keys	El proveedor decide la rotación; el cliente no accede a las claves ni las usa para otros contenidos. Máxima eficiencia, mínimo control.

Ojo con la ubicación de los datos: distintas regiones/países aplican distintas regulaciones.

7.3.3. Encriptación in-transit

- Encriptar **antes** de mover (ej. cifrar un archivo con GPG antes de mandarlo por mail).
- Usar **protocolos de transporte** que aseguren cifrado: TLS (HTTPS), SMTP sobre TLS, IPSEC, SSH. (Acuerdan clave por asimétrica y luego cifran todo el tráfico de forma simétrica.)

7.3.4. Enforcement (Execution-time)

Técnica más usada para monitorear/controlar cómo se transmite la información entre componentes. Se implementa a nivel **aplicativo** (buenos principios de seguridad, no exponer secretos) o a nivel de **infraestructura** con **DLP** (*Data Loss Prevention*). Requiere entender el movimiento de datos (entre procesos, niveles de seguridad o estados).

7.3.5. Encriptación en memoria (TME / TME-MK)

En ambientes con múltiples VMs accediendo a la misma memoria hay riesgo de acceder a información confidencial. Intel (y otros) implementaron en hardware:

- **TME** (*Total Memory Encryption*): **una sola clave** compartida por todos los procesos. La desenscriptación se hace **dentro de la CPU, antes del caché** (AES-XTS en los controladores DRAM/NVRAM).
- **TME-MK** (*Multi-Key*): **múltiples claves**, con control de acceso soportado por los registros de **VT-x** (cada VM usa un *KeyID*).

7.3.6. SoC / IoT / Embedded Security

Escenario de los más desfavorables: chips expuestos físicamente, arquitectura abierta, **debug** accesible, conectividad **wireless**, gran **escala** y **riesgo físico** (tampering). Se recomienda incluir un HSM embebido (ej. chip ATECC608A). Capas: Perception, Application, Network.

7.4. Cumplimiento, detección y respuesta

7.4.1. Comply (Cumplimiento)

“No basta con ser bueno, también hay que parecerlo.”

— dicho en clase Hay que **demostrar** (ej. ante una auditoría) que la protección efectivamente ocurre $\Rightarrow$ todo se **loguea** y queda evidencia.

Retención: guardar información que ya no es necesaria, por temas regulatorios / legales / forenses. Ejemplos: HIPAA **6 años** desde el último uso; SOX **7 años**. Algunas leyes obligan a **rotar las claves de encriptación** (ej. cada 10 años) y re-encriptar todo.

7.4.2. Detect – DLP (Data Loss Prevention)

Herramientas que ayudan a clasificar, inventariar y detectar datos en todos sus estados. **No son preventivas ni correctivas, sino DETECTIVAS:** se activan al fallar la aplicación de una política (generan alerta). Son **complejos de implementar**; hoy sus funcionalidades vienen incorporadas en productos de uso diario (pueden detectar, ej. encriptación no autorizada típica de *ransomware*).

7.4.3. Respond (Respuesta a incidentes)

Equipos dedicados con protocolo. **Primeras 24 h:** registrar fecha/hora, alertar, asegurar el lugar, **frenar la exfiltración**, documentar todo, entrevistar a los involucrados, traer equipo **forense**, notificar a las autoridades. **Más allá de 24 h:** trabajar con forenses, identificar obligaciones legales, reportar a la dirección, identificar problemas futuros y **arreglar el problema**.

! Importante — remarcado en clase

No ocultar el incidente, pero comunicar **solo lo mínimo y necesario**. Si hay regulación de datos personales de por medio, hay que informar al organismo (y, según el caso, a bancos y clientes afectados).

7.5. Encriptación Homomórfica

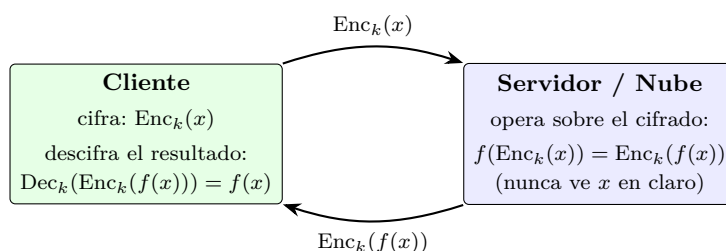
Definición: Encriptación homomórfica

Esquema de cifrado que permite **operar sobre los datos cifrados sin descifrarlos**. Formalmente, para una función f :

$$f(\text{Enc}_k(x)) = \text{Enc}_k(f(x)).$$

Es decir, operar sobre el texto cifrado y luego descifrar da el mismo resultado que operar sobre el texto plano.

Utilidad. Permite **computación en la nube preservando la privacidad**: un tercero (proveedor) puede procesar/analizar datos cifrados **sin verlos nunca** en claro. Útil cuando se quiere delegar cómputo sobre datos sensibles sin exponerlos.



Encriptación homomórfica: el cliente cifra, el servidor opera sobre el texto cifrado sin descifrarlo y el cliente descifra el resultado.

△ Cuidado — error típico

La diapositiva del teórico escribe la propiedad como $f(x) = y \Rightarrow f(\text{Enc}(x)) = f(\text{Dec}(y))$, pero la formulación conceptual correcta y la que conviene recordar es $f(\text{Enc}_k(x)) = \text{Enc}_k(f(x))$: **operar sobre lo cifrado equivale a cifrar el resultado de operar sobre lo plano.**